

TOWER DEFENSE

Rapport de Projet

Alexandre TECHER - Enzo Chadeville
Année : 2025

Résumé :

Ce rapport détaille la conception, l'architecture, les choix techniques et la mise en œuvre de notre jeu Tower Defense développé en C++ avec SFML.

Polytech Dijon

22 décembre 2025

Table des matières

1	Introduction	3
1.1	Contraintes	3
1.2	Objectifs du projet	4
1.2.1	Inspiration	4
1.2.2	Mécanique de gameplay choisi	5
2	Planning prévisionnel	7
2.1	Respect du planning	8
2.2	Tableau comparatif prévu / réalisé	9
3	Évolution du système d’ennemis	10
3.1	Premiers prototypes : formes simples et déplacement minimaliste	10
3.2	Introduction de la grille et du pathfinding A*	11
3.3	Refactorisation : un système généralisé et modulaire	11
3.4	Ajout des ennemis spéciaux : TargetEnemy et FlyEnemy	11
3.5	Version finale du système d’ennemis	12
3.6	Création des ennemis	12
3.7	Types d’ennemis	13
3.8	La mort de l’ennemi	13
3.9	L’ update des ennemis « classiques »	14
3.10	Spécificités des ennemis spéciaux : FlyEnemy , TargetEnemy	15
3.11	Conclusion	17
4	A*	18
4.1	Introduction	18
4.2	Classe Pathfinding AStar	18
4.3	Structure de la grille	19
4.4	Création des noeuds	19
4.5	Association d’un ennemi à un noeud	19
4.6	Heuristique	20
4.7	Recherche de chemin : SolveAStar	20
4.7.1	1. Détermination du noeud de départ	20
4.7.2	2. Recherche du chemin pour chaque noeud d’arrivée	20
4.7.3	3. Reconstruction du chemin	20
4.7.4	4. Sélection du meilleur chemin	20
4.8	Vérification de la possibilité d’un chemin	21
4.9	Mise à jour de la présence d’ennemis	21
4.10	Illustration du fonctionnement global	21
4.11	Conclusion	22
5	Évolution du système de tourelles	23
5.1	Premiers prototypes : formes simples et SFML	23
5.2	Ajout de la portée, du ciblage et du cooldown	23
5.3	Système de masque EnemyMask	24
5.4	Intégration des projectiles et gameplay complet	26
5.5	Création des tourelles	26
5.6	Ciblage et tir	27

5.7	Conclusion	28
6	Évolution du système de projectiles	29
6.1	Construction d'un projectile	30
6.2	Normalisation de la direction	31
6.3	Type de cible et masque <code>EnemyMask</code>	32
6.4	Mise à jour et cycle de vie	33
6.5	Mise à jour du mouvement	33
6.6	Rendu graphique	33
6.7	Gestion de l'état <code>alive</code>	33
6.8	Gestion des collisions	34
6.9	Conclusion	35
7	Game (boucle principale logique)	36
7.1	Génération des ennemis	37
7.2	Génération des tourelles	38
7.3	Destruction des entités	41
7.4	Gestion des vagues	42
7.5	La boucle principale <code>Game::update</code>	44
8	Render	48
8.1	Introduction	48
8.2	Classe Principale : Render	48
8.3	Classe <code>RenderMap</code>	51
8.4	Classe <code>RenderInfo</code>	52
8.5	Classe <code>RenderControl</code>	53
8.6	Classe <code>RenderMenu</code>	54
8.7	Classe <code>RenderGameOver</code>	55
8.8	Classe <code>RenderMainMenu</code>	56
9	Game Event	57
9.1	Introduction	57
9.2	L'énumération <code>AppState</code>	57
9.3	Rôle général de la classe <code>GameEvent</code>	57
9.3.1	Gestion de l'État MAIN MENU	57
9.3.2	Gestion de l'État PLAYING	58
9.3.3	Gestion de l'État MENU (Pause)	58
9.3.4	Gestion de l'État GAME OVER	59
9.3.5	Méthode utilitaire : <code>resetGame</code>	59
10	MAIN	60
10.1	Introduction	60
10.2	Construction des classes	60
10.3	Tant que la fenêtre est ouverte	60
10.4	la fenêtre se ferme	60
11	Conclusion sur l'ensemble du projet	61

1 Introduction

Dans le cadre de notre formation, module Computer Science, il nous a été demandé de réaliser, en C++, une version simplifiée du Tower Defense, jeu emblématique des années 90. Ce travail a été réalisé par Alexandre TECHER et Enzo CHADEVILLE.

1.1 Contraintes

Le projet comporte plusieurs contraintes imposées :

- La carte de jeu doit contenir au moins un ou deux chemins possibles, ainsi que des zones ouvertes permettant le placement de tours
- Le terrain doit être conçu de manière à ce que le joueur puisse réfléchir à des stratégies visant à allonger le parcours des ennemis
- Si tous les chemins menant aux ressources sont bloqués, alors les ennemis ignorent les tourelles
- Le programme doit être développé en programmation orientée objet (Ennemis, Tours, Carte, ...).
- Le programme doit inclure un algorithme de « chemin le plus court »

1.2 Objectifs du projet

1.2.1 Inspiration

Pour notre Tower Defense, nous allons nous inspirer (au niveau du gameplay et une partie de l'HUD) de Tower Wars : StarCraft 2, un mode personnalisé créé dans l'éditeur de cartes de Starcraft 2.

Nous nous inspirons également de Age Of empire pour son HUD des ressources.

Nous avons d'abord regardé Defence Grid. Ce jeu publié en 2008, comporte des chemins prédéfinis pour les ennemis.



FIGURE 1 – Defence Grid.

Limiter les ennemis à certaines directions et le joueur à certaines positions de tours, risquer de créer une répétition dans le jeu.

Contrairement à ce jeu, Tower Wars de Starcraft 2, la map est initialement vierge. Le joueur doit tuer les ennemis en posant des tours mais leurs positions a une influence sur les ennemis. Plusieurs configurations sont possibles et permettent de faire la différence avec nos adversaires.



FIGURE 2 – Tower Wars SC2.

1.2.2 Mécanique de gameplay choisi

Le jeu comporte une carte (map) sur laquelle le joueur pourra créer des tours afin d'éliminer les ennemis qui apparaissent.

La map, de forme rectangulaire et de dimensions (X, Y) , est divisée en trois zones :

- Une zone d'apparition des ennemis ;
- Une zone dans laquelle le joueur peut poser ses tours ;
- Une zone de fin de parcours pour les ennemis.

Un exemple de la disposition des zones sur la carte est présenté à la figure ci-dessous.



FIGURE 3 – Disposition des zones sur la carte.

Les ennemis apparaissent à une position aléatoire dans la zone de départ et doivent atteindre la fin du parcours en utilisant un algorithme de type "chemin le plus court".

Si un ennemi atteint la fin du parcours, le joueur perdra des points de vie.

Pour se défendre, le joueur peut placer des tours qui infligent des dégâts et peuvent également influencer le chemin des ennemis.

La figure 4 illustre l'impact du placement des tours sur la trajectoire des ennemis.

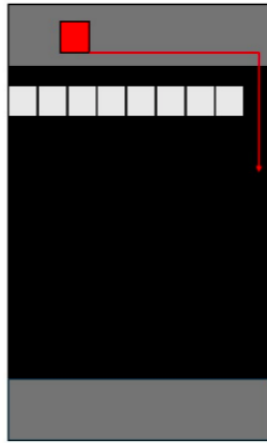


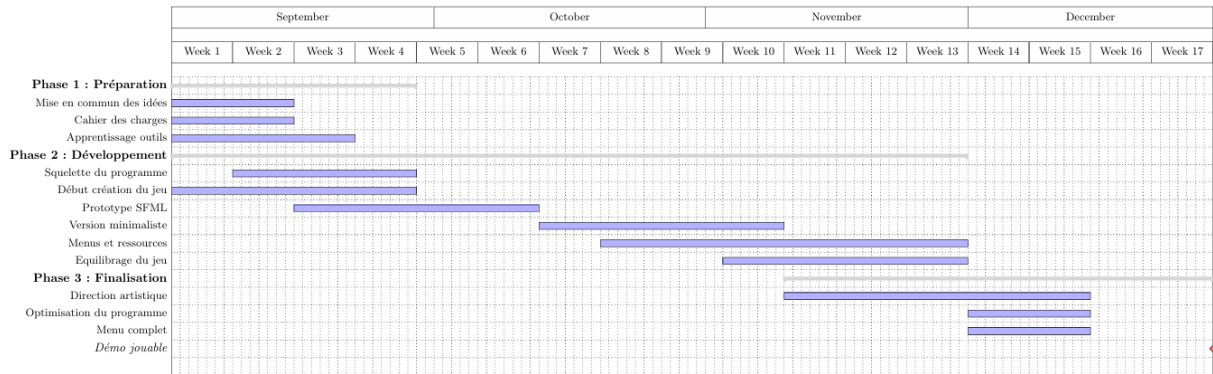
FIGURE 4 – Placement des tours pour dévier le chemin des ennemis.

Ainsi, plus le joueur place judicieusement ses tours, plus le chemin des ennemis s'allongera, augmentant les chances de les éliminer avant qu'ils n'atteignent la fin de la carte.

2 Planning prévisionnel

Cette section présente le **planning prévisionnel** établi en début de projet (semaines 36 à 52). Il sert de référence pour comparer l'avancement réel du projet et vérifier si les objectifs ont été respectés.

Planning prévisionnel (semaines 36 à 52)



2.1 Respect du planning

Dans l'ensemble, nous pouvons dire que nous ne sommes pas parvenus à respecter le planning prévisionnel annoncé en début de projet. À l'origine, nous avions prévu une répartition du travail semaine par semaine, en avançant section par section. Cependant, plusieurs contre-temps sont venus perturber cette organisation : des révisions plus longues que prévu, d'autres projets jugés prioritaires car potentiellement terminables avant le retour en entreprise, et une charge de travail globale mal évaluée.

Nous avons pourtant pris une bonne avance durant les semaines 38 et 39 : apprentissage des outils, rédaction du cahier des charges, mise en commun des idées pour avoir une vision claire et commune du projet. À ce stade, nous disposions déjà d'un squelette solide ainsi que d'une première partie du prototype SFML. D'une certaine manière, nous avons environ trois semaines d'avance par rapport au planning initial. Mais les imprévus se sont accumulés, et nous n'avons quasiment pas pu avancer sur le projet pendant près d'un mois. Nous avons aussi sous-estimé la véritable charge de travail nécessaire pour obtenir une version minimaliste mais fonctionnelle du jeu.

Nous nous sommes donc replongés sérieusement dans le projet lors des semaines 42 et 43, en redéfinissant nos objectifs : se concentrer sur un jeu jouable et respecter au minimum le cahier des charges imposé. C'est à cette période que nous avons terminé le système de pathfinding, ce qui a été une véritable source de motivation pour continuer le développement.

La semaine 44 marque notre retour en entreprise. Les examens étant terminés et la plupart des autres projets bouclés, nous avons enfin pu nous consacrer pleinement au développement du Tower Defense. De la semaine 44 à la semaine 47, nous avons concentré tous nos efforts sur le projet et avons finalement réussi à terminer le jeu. Nous voulions absolument rattraper notre retard et même prendre de l'avance sur les projets restants ainsi que sur les futures révisions. Durant cette période, nous avons travaillé pratiquement tous les jours, autant que possible. Cette intensité de travail nous a permis de terminer le jeu avec environ trois semaines d'avance.

Pour conclure, nous avons sous-estimé le temps que demandaient certaines tâches du Tower Defense. Le projet a pris plus de temps que prévu, en partie à cause de notre manque d'expérience en C++, de nos disponibilités variables et de notre volonté de bien faire. Malgré cela, nous avons réussi à rendre un projet complet et fonctionnel dans les temps, même si le temps consacré chaque semaine a été très inégal.

2.2 Tableau comparatif prévu / réalisé

Le tableau ci-dessous résume les principales phases du projet avec les périodes prévues et les périodes réelles.

Phase	Tâches principales	Prévu	Réalisé	Commentaire
Préparation	Mise en commun des idées, cahier des charges, apprentissage des outils	Semaines 36–40	Semaines 38–39	Avance prise grâce à une bonne organisation de départ.
Développement (cœur du jeu)	Squelette du programme, prototype SFML, pathfinding, ennemis de base	Semaines 40–47	Semaines 42–47	Pause d'environ un mois à cause des examens et autres projets.
Finalisation	Direction artistique, équilibrage, menus, version jouable complète	Semaines 48–52	Semaines 44–47	Accélération importante du travail; projet terminé environ 3 semaines avant la date prévue.

TABLE 1 – Comparatif entre planning prévisionnel et réalisation effective.

3 Évolution du système d'ennemis

Cette partie se concentre sur les ennemis : leur évolution au cours du projet, les choix de conception que nous avons faits, ainsi que quelques limites et pistes d'amélioration.

Avant d'arriver à la version finale présentée dans ce rapport, le système d'ennemis est passé par plusieurs prototypes. L'objectif de cette section est de montrer comment on est parti d'ennemis « ronds qui avancent tout droit » pour arriver à un module cohérent, modulaire et suffisamment riche pour servir de base au gameplay.

3.1 Premiers prototypes : formes simples et déplacement minimaliste

Au début du projet, les ennemis n'étaient rien de plus que des cercles dessinés dans la fenêtre SFML. Aucun comportement avancé : pas de pathfinding, pas de type d'ennemi, aucune interaction avec les tourelles. L'objectif était surtout de valider les bases :

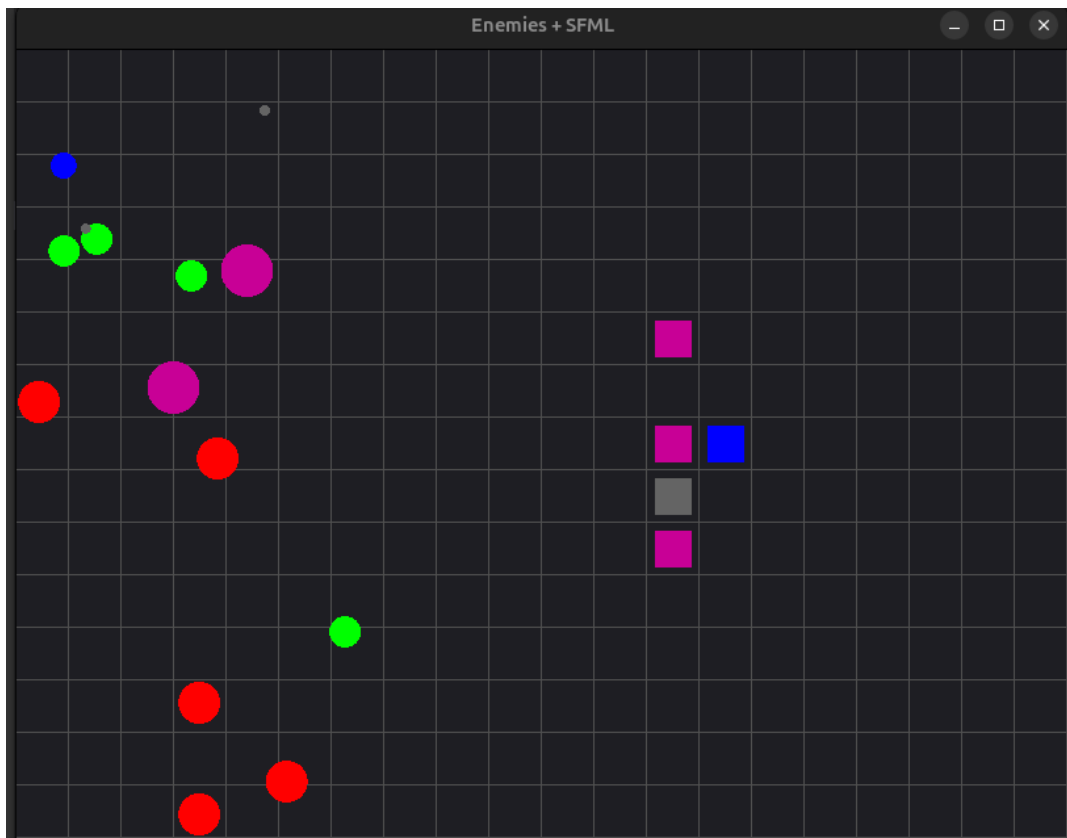


FIGURE 5 – Premiers prototypes sous SFML : affichage des ennemis en cercles et utilisation d'une grille pour faciliter le développement.

- instancier plusieurs ennemis en même temps ;
- gérer leur affichage ;
- leur attribuer des statistiques de base (vie, vitesse, taille) ;
- les faire apparaître à l'écran via quelques classes simples (`Basic`, `Fast`, `Tank...`).

Ces premiers essais nous ont surtout permis de nous familiariser avec l'héritage, les pointeurs d'objets polymorphes et la gestion de tableaux dynamiques d'ennemis. À ce stade, chaque type d'ennemi ne se distinguait que par ses valeurs de construction.

3.2 Introduction de la grille et du pathfinding A*

La deuxième étape a été d'introduire une grille et un système de pathfinding. Nous voulions que les ennemis suivent un chemin lisible sur la carte, et que le joueur puisse déformer ce chemin en posant des tourelles.

L'algorithme A* a été adapté à notre grille SFML pour calculer un chemin « le plus court » entre la zone de spawn et la zone d'arrivée. Chaque ennemi suit alors une suite de points de pathfinding (`pathPoints`) recalculée régulièrement.

Cette première intégration fonctionnait, mais avec plusieurs limites :

- recalculs de chemins très fréquents, donc une charge inutile ;
- logique de spawn / mort encore dispersée dans le code ;
- aucun comportement offensif : les ennemis se contentent d'avancer ;
- les tourelles n'ont aucun impact sur l'IA, en dehors du simple blocage de cellule.

Cela reste néanmoins la première version vraiment jouable du système d'ennemis. A posteriori, ce choix d'A* « brut » est efficace pour une petite grille, mais assez coûteux et peu flexible dès qu'on multiplie les ennemis ou les recalculs (voir section A* pour plus de détails).

3.3 Refactorisation : un système généralisé et modulaire

Au fil du projet, la classe `Enemy` a été entièrement restructurée pour éviter l'empilement de cas particuliers. L'objectif était d'obtenir :

- un constructeur générique pour tous les ennemis ;
- une gestion claire de la mort (`dead`) et de l'arrivée au but (`reachedGoal`) ;
- une énumération `EnemyKind` pour identifier les types ;
- un masque d'autorisations pour les interactions tourelles / ennemis ;
- une fonction `update()` générique pour les ennemis « classiques ».

Cette refactorisation a servi de base pour intégrer ensuite des comportements plus complexes (ennemis volants, chasseurs de tourelles, tirs, etc.), sans tout recoder à chaque nouveau type.

3.4 Ajout des ennemis spéciaux : `TargetEnemy` et `FlyEnemy`

Une fois les fondations en place, nous avons introduit des ennemis avec un rôle plus spécifique : attaquer directement les tourelles du joueur.

Ces ennemis spéciaux ne suivent plus A* comme les autres. Un `FlyEnemy` par exemple :

- cherche la tourelle la plus proche (`acquireTarget()`) ;
- se déplace directement vers elle (`moveTowards()`) ;
- s'arrête lorsqu'il est à portée et tire (`tryShoot()`) ;
- avance simplement vers la droite s'il n'a aucune cible.

Ce choix permet de créer un deuxième type de menace : non seulement les ennemis veulent atteindre la base, mais certains cherchent aussi à détruire activement les défenses du joueur.

3.5 Version finale du système d'ennemis

À l'état actuel, le module `Enemy` fournit :

- un **constructeur générique** pour instancier rapidement de nouveaux types ;
- une **énumération** `EnemyKind` pour identifier proprement chaque catégorie ;
- des **règles de vie / mort simples** via les booléens `dead` et `reachedGoal` ;
- un **pathfinding A*** pour les ennemis terrestres ;
- une **IA de poursuite et de tir** pour les ennemis aériens ;
- une **intégration complète aux tourelles** via les masques, les projectiles et `tryShoot()`.

Le système reste perfectible (coût des recalculs A*, code parfois un peu verbeux), mais il est assez modulaire pour ajouter de nouveaux ennemis sans casser l'architecture existante.

3.6 Création des ennemis

```
1 // Constructeur générique d'un ennemi
2 Enemy(int hp, float spd, float radius,
3       const std::string& texturePath,
4       int resourceValue);
```

Lorsqu'on construit un ennemi, on lui fournit notamment :

- ses points de vie (HP) ;
- sa vitesse de déplacement ;
- son rayon de collision (taille en jeu) ;
- le chemin vers sa texture ;
- la quantité de ressources gagnée à sa mort.

Chaque classe dérivée (`BasicEnemy`, `FastEnemy`, `TankEnemy`, `TargetEnemy`, `FlyEnemy`) ne fait qu'appeler ce constructeur générique avec un jeu de paramètres différents. Cela permet de décrire des profils variés :

- **BasicEnemy** : profil « standard » (HP moyens, vitesse moyenne, taille normale) ;
- **FastEnemy** : peu de vie, très rapide et difficile à intercepter ;
- **TankEnemy** : énormément de PV, lent et très volumineux ;
- **TargetEnemy** : proche du `BasicEnemy`, mais capable de cibler les tourelles ;
- **FlyEnemy** : peu de PV, assez rapide, orienté destruction de tourelles et immunisé à la plupart des tourelles classiques.

Cette petite « palette » d'ennemis suffit déjà à rendre les vagues différentes et à forcer le joueur à adapter ses choix de tourelles.

Difficulté rencontrée

Au départ, nous voulions tout définir dans les `.hpp` (y compris la configuration des ennemis). En pratique, la taille des ennemis dépend de la taille des cellules de la grille, qui est calculée dans `Render`. Cela a rapidement mené à des problèmes de compilation (dépendances circulaires, ordre de définition, etc.).

Nous aurions pu multiplier les *forward declarations*, mais cela compliquait inutilement l'architecture. Nous avons donc déplacé la logique d'initialisation complète des ennemis dans les `.cpp`, ce qui :

- réduit le couplage entre modules ;
- clarifie la séparation interface / implémentation ;
- évite d'empiler des déclarations anticipées peu lisibles.

3.7 Types d'ennemis

```
1 enum class EnemyKind { Basic = 0, Fast, Tank, Target, Fly };
```

`EnemyKind` nous permet d'identifier le type d'un ennemi sans utiliser de `dynamic_cast`. Le choix d'une *enum class* évite les conflits de noms : on peut avoir `EnemyKind::Basic` et `TourelleKind::Basic` sans ambiguïté.

L'énumération est fournie au constructeur de chaque ennemi et utilisée partout où l'on a besoin de filtrer un type, par exemple côté tourelles :

```
1 if (enemy->getKind() == EnemyKind::Fast) { /* ... */ }
```

Elle devient surtout très utile combinée au système de masques (`EnemyMask`) côté tourelles et projectiles, qui repose entièrement sur ces types logiques.

3.8 La mort de l'ennemi

L'état de vie d'un ennemi est résumé par le booléen `dead`. Tant qu'il vaut `false`, l'ennemi est mis à jour (déplacement, pathfinding, dessin, etc.). Il passe à `true` uniquement lorsque ses points de vie tombent à zéro :

```
1 void Enemy::takeDamage(int amount) {
2     if (dead) return;
3     health -= amount;
4     if (health <= 0) {
5         health = 0;
6         dead = true;
7     }
8 }
9
10 void Enemy::draw(sf::RenderTarget& win) const {
11     if (!dead) win.draw(sprite);
12 }
```

L'idée est simple : `dead` agit comme un masque logique. Dès qu'un ennemi est mort, toutes les opérations coûteuses (recalcul de chemin, dessin, etc.) sont court-circuitées. On retrouve la même philosophie dans d'autres modules (`alive` pour les projectiles, `reachedGoal` pour les ennemis qui ont atteint la fin...).

3.9 L'update des ennemis « classiques »

Pour la majorité des ennemis au sol, nous utilisons une fonction générique `Enemy::update` qui s'appuie sur A* pour le déplacement.

La logique est la suivante :

1. Si l'ennemi est mort, on ne fait rien ;
2. SI le chemin est vide ou obsolète (`needRepath`), on appelle `SolveAStar` ;
3. On avance progressivement vers le prochain point du chemin ;
4. Si tous les points ont été atteints, on passe `reachedGoal` à `true`.

En pseudo-code simplifié :

```
1 // Recalcule le chemin si besoin
2 if (pathPoints.empty() || needRepath
3     || currentPathIndex >= (int)pathPoints.size())
4 {
5     pathPoints          = pathfinder.SolveAStar(getPosition());
6     currentPathIndex = 0;
7     needRepath          = false;
8 }
9
10 // Avance le long du chemin
11 if (currentPathIndex < (int)pathPoints.size()) {
12     sf::Vector2f pos      = getPosition();
13     sf::Vector2f target = pathPoints[currentPathIndex];
14     sf::Vector2f dir      = target - pos;
15     float dist = std::sqrt(dir.x*dir.x + dir.y*dir.y);
16     float step = getSpeed();
17
18     if (dist <= step || dist == 0.f) {
19         setPosition(target.x, target.y);
20         ++currentPathIndex;
21     } else {
22         dir /= dist;          // normalisation
23         pos += dir * step;
24         setPosition(pos.x, pos.y);
25     }
26 }
27
28 // Fin du chemin : l'ennemi atteint le goal
29 if (currentPathIndex >= (int)pathPoints.size()) {
30     reachedGoal = true;
31 }
```

Lorsqu'un ennemi atteint le but, `reachedGoal` joue le même rôle que `dead` : il sort des mises à jour courantes et inflige des dégâts au joueur (logique gérée côté `Game`).

3.10 Spécificités des ennemis spéciaux : FlyEnemy, TargetEnemy

Certains ennemis ne suivent pas le schéma « A* + goal ». C'est le cas de FlyEnemy (et dans une moindre mesure TargetEnemy), dont le rôle est de s'attaquer directement aux tourelles.

Mouvement dirigé : moveTowards()

Plutôt que de suivre un chemin discret, FlyEnemy se déplace en ligne droite vers une position cible :

```
1 void FlyEnemy::moveTowards(const sf::Vector2f& dest, float step) {
2     sf::Vector2f p = getPosition();
3     sf::Vector2f v = dest - p;
4     float L = std::sqrt(v.x*v.x + v.y*v.y);
5     if (L < 1e-4f) return;
6     v /= L;          // normalisation
7     p += v * step;    // déplacement
8     setPosition(p.x, p.y);
9 }
```

Ce choix est cohérent avec notre carte : peu d'obstacles fixes, et des tourelles qui sont justement la cible de ce type d'ennemi. Repasser par A* n'apporterait pas grand-chose ici et alourdirait le code.

Ciblage des tourelles : acquireTarget()

Le ciblage repose sur une recherche de la tourelle la plus proche en distance euclidienne au carré :

```
1 void FlyEnemy::acquireTarget() {
2     target = nullptr;
3     if (!towers) return;
4
5     float bestD2 = std::numeric_limits<float>::max();
6     sf::Vector2f pos = getPosition();
7
8     for (auto* t : *towers) {
9         if (!t || t->isDestroyed()) continue;
10        sf::Vector2f d = t->getPosition() - pos;
11        float d2 = d.x*d.x + d.y*d.y;
12        if (d2 < bestD2) {
13            bestD2 = d2;
14            target = t;
15        }
16    }
17 }
```

Le recalcul est volontairement fait à chaque update. Cela compense le fait que le déplacement est purement géométrique : si le joueur pose une nouvelle tourelle plus proche, l'ennemi change immédiatement de cible.

Gestion du tir : tryShoot()

Le tir des FlyEnemy suit la même philosophie que celui des tourelles, mais dans l'autre sens (projectiles vers les tours) :

```
1 bool FlyEnemy::tryShoot(float dt, std::vector<Projectile>& out) {
2     if (!target || target->isDestroyed()) return false;
3
4     sf::Vector2f pos  = getPosition();
5     sf::Vector2f tpos = target->getPosition();
6     sf::Vector2f d     = tpos - pos;
7     float d2 = d.x*d.x + d.y*d.y;
8
9     if (d2 > shootRange * shootRange) return false;
10
11     fireCooldown -= dt;
12     if (fireCooldown > 0.f) return false;
13
14     out.emplace_back(
15         pos, tpos, projSpeed, projDamage,
16         4.f, 0u, Projectile::TargetType::Towers
17     );
18
19     fireCooldown = 1.f / fireRate;
20     return true;
21 }
```

Encore une fois, on retrouve un schéma très proche de celui des tourelles : test de portée, gestion d'un cooldown, création d'un projectile. La différence principale réside dans la cible (Projectile::TargetType::Towers) et l'utilisation de masques allégés côté tours.

Update spécifique de FlyEnemy

L'update des ennemis spéciaux combine ces briques :

- recherche ou mise à jour de la cible (acquireTarget());
- déplacement vers la tourelle si une cible existe, sinon avance vers la droite;
- détection de l'arrivée en bord de carte (reachedGoal);
- tir via tryShoot() (géré côté Game).

La boucle comportementale est volontairement simple : se rapprocher jusqu'à être en portée, s'arrêter, puis tirer tant que la tourelle est en vie. Ce comportement rend FlyEnemy particulièrement dangereux pour les défenses mal protégées.

3.11 Conclusion

Le module `Enemy` est l'un des piliers de notre Tower Defense. Il essaie de concilier trois objectifs :

- **simplicité d'implémentation** : un constructeur générique, quelques booléens, pas de hiérarchie trop profonde ;
- **lisibilité** : états clairement séparés (`dead`, `reachedGoal`), comportements spéciaux isolés dans des classes dédiées ;
- **richesse de gameplay** : plusieurs profils d'ennemis, des menaces différentes (course au goal, destruction de tourelles).

Dans l'ensemble, le système fonctionne bien à l'échelle de notre projet, mais il montre aussi ses limites : dépendance forte à A^* , recalculs fréquents, code parfois un peu lourd pour des actions simples. Avec plus de temps, nous pourrions :

- mieux mutualiser certaines parties du code (déplacement / tir) ;
- réduire le coût des recalculs A^* (mise en cache, fréquence limitée) ;
- introduire des comportements plus variés sans multiplier les `if` partout.

Malgré ces points, la structure actuelle offre une base solide pour ajouter de nouveaux types d'ennemis ou faire évoluer les comportements sans remettre en cause tout le système.

4 A*

4.1 Introduction

Afin de permettre aux ennemis de se déplacer de manière intelligente sur la carte, un algorithme de chemin le plus est nécessaire. Dans un premier temps, une lecture de la description de la librairie `boost_graph` a permis de cerner les algorithmes de pathfinding disponibles.

lien de description : <https://www.boost.org/doc/libs/latest/libs/graph/doc/index.html>

L'algorithme Dijkstra's Shortest Paths a été envisagé. Mais après réflexion et comparaison sur wikipédia, l'algorithme A* (A-star) a été préféré car il est généralement plus "efficace" pour les jeux vidéo et les applications en temps réel. Il est pour ainsi dire plus utilisé.

4.2 Classe Pathfinding AStar

La classe Pathfinding AStar implémente l'algorithme A* pour trouver le chemin le plus court entre deux points sur la carte. Elle est utilisée par la classe Enemy pour déterminer le chemin à suivre.

Tout d'abord, le programme de A* est inspiré du github suivant :

https://github.com/OneLoneCoder/Javidx9/blob/master/ConsoleGameEngine/SmallerProject/OneLoneCoder_PathFinding_AStar.cpp

La vidéo suivante explique le fonctionnement de l'algorithme A* :

<https://www.youtube.com/watch?v=icZj67PTFhc>

Après modification du code source pour une utilisation avec SFML, l'algorithme A* une première adaptation nous a permis d'obtenir le résultat suivant :

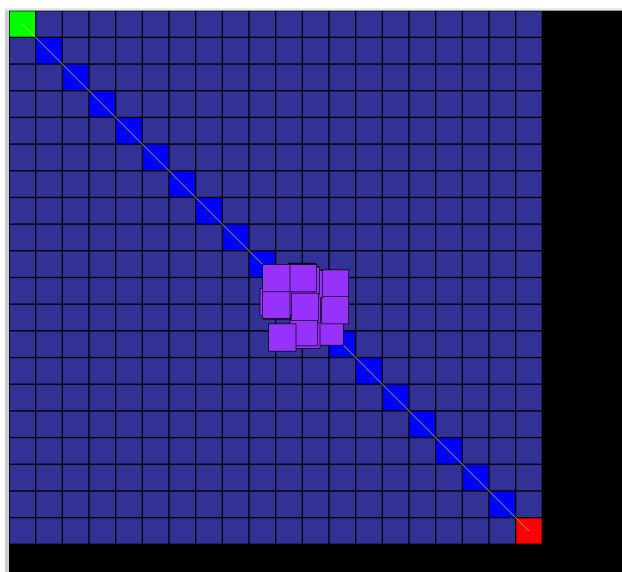


FIGURE 6 – Déplacement des ennemis avec l'algorithme A*

Le chemin le plus court (trouvé) est représenté par une ligne jaune reliant les nœuds verts

(noeud de départ) et rouge (noeud d'arrivée). Les noeuds bleus claires représentent les noeuds libres visités par l'algorithme, tandis que les noeuds bleus foncés représentent les noeuds libres non visités.

Ici, seulement un noeud est vert, car on affiche le noeud de départ du premier ennemi apparu.

4.3 Structure de la grille

Nous avons décidé de séparer les calculs et la représentation des cellules. Ainsi les caractéristiques de la grille permet de définir un ensemble de noeuds (**Nodes**). Chaque noeud représente une cellule et contient :

- sa position (**x,y**) en coordonnées pixel,
- sa position dans la grille (**row,col**),
- un indicateur **turreted** (présence de tourelle),
- un indicateur **EnemyPresence**,
- deux valeurs de coût : **fLocalGoal** et **fGlobalGoal**,
- une liste de voisins directs (8 directions possibles),
- un pointeur vers son parent dans le chemin reconstruit.

On suppose que **fLocalGoal** et **fGlobalGoal** sont infini initialement (inconnu).

Le A* va devoir changer ces données pour trouver le chemin le plus court.

4.4 Création des noeuds

Tout comme **buildCells**, nous avons **CreateNodes** pour l'initialisation des noeuds. La méthode **CreateNodes** reçoit un vecteur de cellules graphiques :

1. Elle détermine la taille de la grille en fonction des indices **row** et **col**.
2. Chaque cellule est convertie en un noeud de navigation.
3. Les voisins d'un noeud sont tous ceux qui respectent ces deux conditions :

$$|row_i - row_j| \leq 1 \quad \text{et} \quad |col_i - col_j| \leq 1.$$

Pour augmenter les possibilités de déplacement des ennemis nous avons modifié le nombre de noeud de fin. Précédemment un seul noeud servait d'arrivée. Nous avons étendu ses caractéristique à l'ensemble de la dernière colonne de jeu et stocké les noeuds dans une liste **nodeEnd**.

A* est un algorithme de recherche de chemin le plus court. Cependant il peut fournir un chemin mais pas le plus court, son but étant aussi d'en trouver un le plus rapidement possible. Dans notre cas, considérer plusieurs fin par **NodeEnd** évite ce problème.

4.5 Association d'un ennemi à un noeud

Pour effectuer nos calcul nous convertissons la position de l'ennemi en un noeud. Nous définissons la méthode **findClosestNode** qui associe à une position le noeud dont le centre est le plus proche. Il paraissait logique de gérer le chemin seulement avec des noeuds. d'où la conversion des positions des ennemis en noeuds.

4.6 Heuristique

L'heuristique est un moyen de résoudre des problèmes avec des données non complètes. Dans notre cas on l'utilise dans l'algorithme A pour calculer la distance euclidienne entre deux noeuds :

$$h(a, b) = \sqrt{(a_x - b_x)^2 + (a_y - b_y)^2}.$$

Elle permet d'estimer la distance restante entre le noeud ennemi courant et un noeud de fin.

4.7 Recherche de chemin : SolveAStar

La méthode **SolveAStar** calcule le chemin optimal selon les étapes suivantes :

4.7.1 1. Détermination du noeud de départ

Le noeud de départ est celui le plus proche de **startPos**. Cette position réelle est ensuite insérée en tête du chemin final.

Pour éviter que les ennemis se focalise sur un noeud de fin, nous cherchons le chemin le plus court mais avec tous les noeuds de fin.

4.7.2 2. Recherche du chemin pour chaque noeud d'arrivée

Pour chaque noeud cible dans **nodeEnd**, l'algorithme A est exécuté :

1. Tous les noeuds sont réinitialisés (coûts et visite).
2. Le coût local du noeud de départ est mis à 0.
3. Le coût global est initialisé par l'heuristique.
4. Une liste de noeuds non testés est maintenue, triée selon leur **fGlobalGoal**.
5. À chaque itération :
 - (a) le noeud au plus faible coût global est sélectionné,
 - (b) ses voisins non visités et non bloqués par une tourelle sont analysés,
 - (c) un coût d'étape est calculé, prenant en compte la distance de déplacement et un éventuel facteur de pénalité,
 - (d) si ce coût permet d'améliorer le chemin vers un voisin, les valeurs **fLocalGoal**, **fGlobalGoal** et **parent** sont mises à jour.

4.7.3 3. Reconstruction du chemin

Si un noeud d'arrivée est atteint :

- on remonte la chaîne des **parent**,
- le chemin est inversé pour être dans l'ordre départ → arrivée,
- il est ajouté à la liste **paths**.

4.7.4 4. Sélection du meilleur chemin

Si plusieurs noeuds d'arrivée étaient accessibles, plusieurs chemins sont générés. Pour chacun, un coût total leur est associé.

Le chemin ayant le coût le plus faible est sélectionné pour la trajectoire de notre ennemi.

4.8 Vérification de la possibilité d'un chemin

La méthode `IsPathStillPossible` vérifie, sans A, qu'au moins un chemin existe encore entre un noeud de départ et un noeud d'arrivée. Elle utilise une exploration simple s'arrêtant dès qu'un noeud de fin est atteint.

Cette fonction sera utilisée dans les conditions de positionnement de tourelle. Tant qu'il existe au moins un chemin, la tourelle peut être placée. Sinon on bloque la possibilité au joueur.

4.9 Mise à jour de la présence d'ennemis

La méthode `updateEnemyPresence` marque les noeuds sur lesquels un ennemi se trouve physiquement. Pour cela, elle teste la distance entre l'ennemi et le centre de chaque noeud, et applique un seuil basé sur la taille des cellules.

Sans cette fonction le joueur peut poser une tourelle sur un ennemi qui sera ignorée dans le calcul (car ennemi déjà dans le noeud).

De plus, on élimine le cas d'un ennemi qui traverse les tours.

4.10 Illustration du fonctionnement global

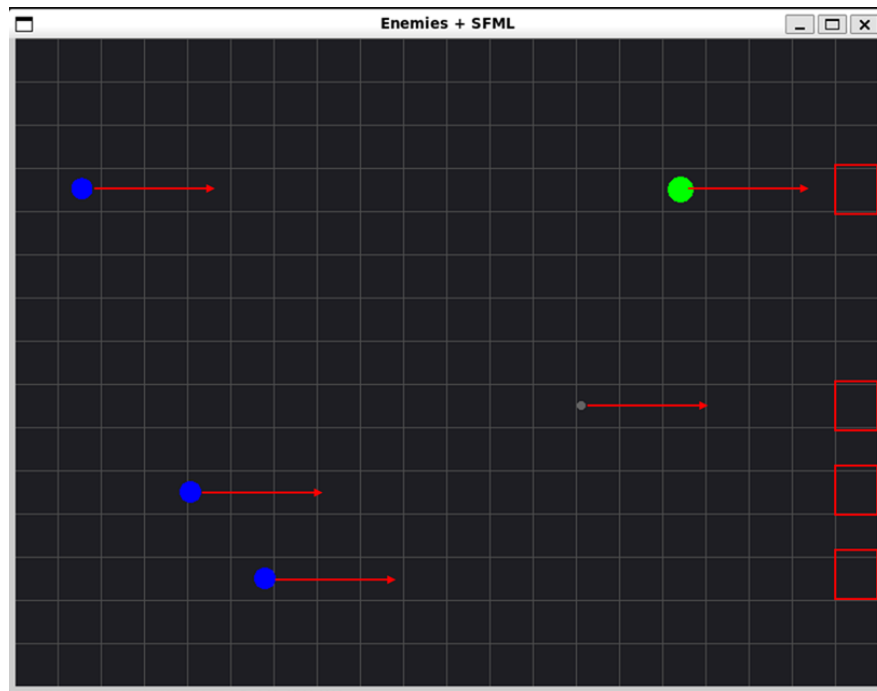


FIGURE 7 – Déplacement des ennemis vers le noeud de fin le plus proche.

A chaque calcul du chemin, la position de l'ennemi est pris en compte pour créer un noeud de départ. Les noeuds d'arrivées sont les cellules de la dernière colonnes. L'ennemi ira vers la cellule de fin la plus proche.

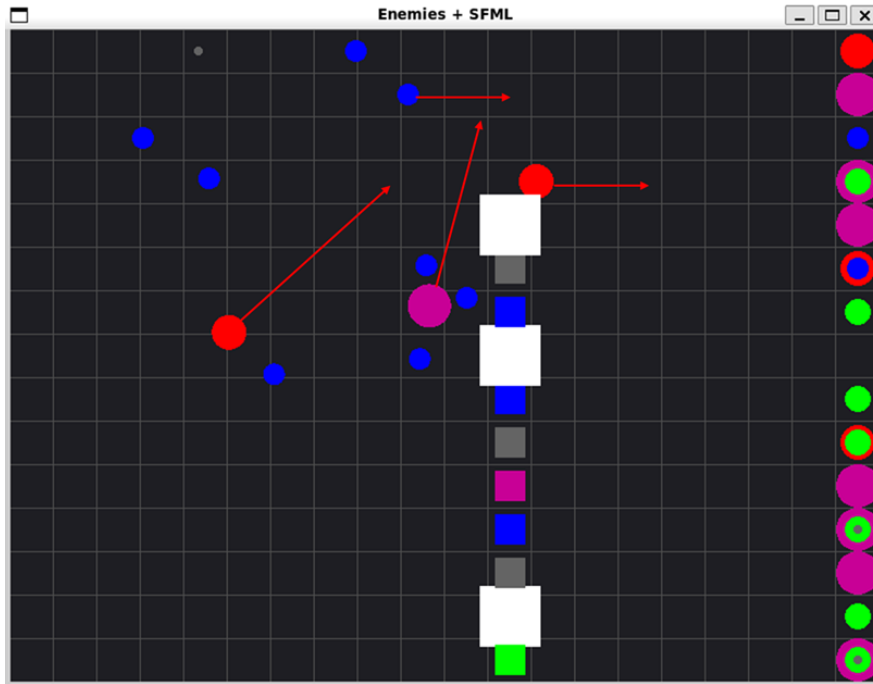


FIGURE 8 – Déviation des ennemis pour éviter les tours.

Lorsque le joueur positionne une tourelle, le calcul du A^* est à nouveau exécuté. Une cellule possédant une tour a son booléen `turreted` à `true`. Cela permet de considérer cette cellule comme bloquée pour le A^* .

De ce fait, les ennemis vont contourner les tours.

4.11 Conclusion

Dans notre utilisation, A^* est un algorithme qui satisfait nos besoins. La déclaration de `Noeuds` permet de séparer la partie affichage avec les cellules et la partie calcul pour le déplacement des ennemis et la vérification d'existence de chemin. Il y a toujours moyen d'améliorer l'algorithme en choisissant par exemple un meilleur heuristique, mais cela demande plus de réflexion et peut être non nécessaire au niveau de notre projet.

5 Évolution du système de tourelles

Cette partie est consacrée au module **Tourelle**, l'un des éléments centraux du gameplay. Son rôle est d'offrir au joueur un moyen automatique de défense : sélectionner, viser et attaquer les ennemis présents sur la carte. Nous présentons ici l'évolution du système, les principaux choix de conception et quelques limites.

La conception des tourelles s'est faite par étapes, au fur et à mesure que le jeu gagnait en complexité.

5.1 Premiers prototypes : formes simples et SFML

Au début du projet, les tourelles n'étaient que de simples carrés placés sur la grille SFML. Elles n'avaient ni portée, ni système de tir, ni gestion de cooldown. L'objectif était surtout de :

- afficher une tourelle sur une case donnée ;
- la stocker dans un tableau dynamique ;
- la différencier des ennemis ;
- rattacher sa position à celle d'une cellule de la grille.

Cette phase a servi de "laboratoire" pour valider deux points techniques : la manipulation des formes SFML et le placement centralisé via la grille.

5.2 Ajout de la portée, du ciblage et du cooldown

La deuxième étape a été l'introduction du trio fondamental : **portée** + **acquireTarget()** + **tryShoot()**.

C'est à ce moment que les tourelles sont devenues de vraies entités de gameplay :

- recherche de la cible la plus proche ;
- calcul de distance en *distance*² pour éviter les racines carrées ;
- gestion de la cadence de tir via un cooldown ;
- création de projectiles "à la volée".

Ce schéma (ciblage + test de portée + cooldown + création de projectile) sera réutilisé plus tard côté ennemis, ce qui simplifie fortement la cohérence globale du jeu.

5.3 Système de masque EnemyMask

Chaque ennemi est associé à un **type** via l'énumération `EnemyKind` :

```
1 enum class EnemyKind { Basic = 0, Fast, Tank, Target, Fly };
```

Ce type permet d'identifier un ennemi sans recourir à `dynamic_cast`. Il est récupéré via `getKind()` et sert de base à notre système de masque d'autorisations côté tourelles.

L'idée est la suivante : chaque tourelle possède un `EnemyMask` qui encode, sous forme de bits, quels types d'ennemis elle est autorisée à viser. Plutôt que d'avoir plusieurs booléens (`canHitBasic`, `canHitFast`, etc.), on regroupe tout dans un entier non signé :

```
1 using EnemyMask = unsigned int;
```

Chaque valeur de `EnemyKind` correspond à un bit dans le masque :

```
1 static constexpr EnemyMask kindBit(EnemyKind k) {  
2     return 1u << static_cast<unsigned int>(k);  
3 }
```

Par exemple :

EnemyKind	valeur	1u « k	binaire	décimal
Basic	0	1u « 0	00001	1
Fast	1	1u « 1	00010	2
Tank	2	1u « 2	00100	4
Target	3	1u « 3	01000	8
Fly	4	1u « 4	10000	16

Un masque `EnemyMask` n'est donc qu'une combinaison de ces bits. Par exemple, `allowedMask = 0b01101` signifie : autorisé sur `Basic`, `Tank` et `Fly`, mais pas sur `Fast` ni `Target`.

Configuration du masque dans Tourelle

Dans `Tourelle`, nous stockons le masque courant dans un membre :

```
1 // Par défaut, tous les bits      1 : autorise tout  
2 EnemyMask allowedMask = ~0u;
```

Quelques fonctions utilitaires permettent de configurer ce masque :

```
1 void allowOnly(EnemyMask m) { allowedMask = m; }  
2  
3 void allowAlso(EnemyKind k) { allowedMask |= kindBit(k); }  
4  
5 void forbid(EnemyKind k)     { allowedMask &= ~kindBit(k); }
```

Enfin, la fonction `accepts(e)` détermine si une tourelle a le droit de tirer sur un ennemi donné :

```
1 bool accepts(const Enemy* e) const {  
2     EnemyKind k = e->getKind();  
3     return (allowedMask & kindBit(k)) != 0;  
4 }
```

Le test se résume donc à : "le bit correspondant au type de cet ennemi est-il activé dans `allowedMask` ?". Ce mécanisme est très compact et reste facilement extensible si de nouveaux types d'ennemis sont ajoutés plus tard.

Exemple : BasicTourelle

La BasicTourelle part avec `allowedMask = 0u` (tout autorisé), puis désactive les volants :

```
1 BasicTourelle(float cellsize) : Tourelle(/*...*/) {
2     name = "Basic";
3     // Autoriser tout SAUF les volants :
4     forbid(EnemyKind::Fly);
5 }
```

Elle peut donc cibler tous les ennemis sauf ceux de type Fly.

Exemple : FlyTourelle

Pour FlyTourelle, c'est l'inverse : elle ne doit tirer que sur les ennemis volants.

```
1 FlyTourelle(float cellsize) : Tourelle(/*...*/) {
2     name = "Fly";
3     forbid(EnemyKind::Basic);
4     forbid(EnemyKind::Fast);
5     forbid(EnemyKind::Tank);
6     forbid(EnemyKind::Target);
7 }
```

À la fin, seul le bit Fly reste à 1, donc `accepts(e)` ne renverra `true` que pour `EnemyKind::Fly`. Ce système de masque reste simple à manipuler tout en couvrant la plupart des cas dont nous avons besoin. À plus grande échelle, une approche plus déclarative (fichiers de configuration, scripts...) serait probablement plus souple, mais ici le compromis lisibilité / contrôle direct est satisfaisant.

5.4 Intégration des projectiles et gameplay complet

Une fois les projectiles intégrés, les tourelles ont pris toute leur dimension stratégique :

- projectiles autonomes (logique de déplacement séparée) ;
- collisions cercle/cercle avec les ennemis ;
- dégâts et comportements spécifiques ;
- variantes de tir (tir unique, “shotgun”, haute cadence, etc.).

Le module `Tourelle` devient alors un “générateur d’attaques” configurable : chaque type de tourelle se distingue essentiellement par ses statistiques (dégâts, portée, cadence, coût, etc.) et par quelques comportements particuliers.

5.5 Création des tourelles

Contrairement aux ennemis, la création des tourelles nous a posé moins de problèmes de dépendances. Nous avons pu conserver la déclaration des paramètres directement dans le `.hpp`, en utilisant des *forward declarations* pour `Enemy` et `Projectile` afin d’éviter les inclusions circulaires.

Dans le `.hpp`, nous définissons les paramètres d’une tourelle : dégâts, portée, cadence de tir, vitesse de projectile, rayon (lié à la taille de cellule), couleur et coût de placement. Chaque type de tourelle est une “template” de statistiques cohérente :

```
1 class BasicTourelle : public Tourelle {
2 public:
3     static int counter;
4     explicit BasicTourelle(float cellsize)
5         : Tourelle(
6             /*damage*/          6,
7             /*range*/           220.f,
8             /*fireRate*/        2.f,
9             /*projectileSpeed*/ 450.f,
10            /*radius*/          cellsize,
11            /*color*/            sf::Color(60, 180, 255),
12            /*cost*/             10
13        )
14    {}
15 };
```

Nous avons ainsi défini plusieurs profils :

- **tourelles Basic** : polyvalentes, dégâts moyens, bon compromis portée / cadence ;
- **tourelles rapides** : forte cadence et projectiles rapides, efficaces contre les petits ennemis ;
- **tourelle Fly** : spécialisée contre les ennemis volants ;
- **tourelle Shotgun** : tir dispersé (plusieurs projectiles en éventail) ;
- **tourelle Poison** : cadence correcte mais dégâts faibles, orientée sur la durée.

L’idée est la même que pour `EnemyKind` : chaque tourelle a un rôle bien identifié, ce qui encourage le joueur à adapter sa composition de défense.

5.6 Ciblage et tir

Les fonctions `acquireTarget()` et `tryShoot()` ont d'abord été développées pour les tourelles : calcul de distance au carré, sélection du meilleur candidat, vérification du masque, gestion du cooldown, puis création d'un projectile.

Ce schéma a ensuite été réutilisé côté ennemis spéciaux (`FlyEnemy`, `TargetEnemy`) en inversant simplement les rôles : cette fois, c'est l'ennemi qui parcourt la liste des tourelles.

Cas particulier : `ShotgunTourelle::tryShoot`

La majorité des tourelles tirent un seul projectile vers la cible. La `ShotgunTourelle`, elle, doit tirer plusieurs projectiles légèrement déviés autour de la direction principale, pour simuler un tir dispersé.

Pour cela, nous redéfinissons `tryShoot()` dans la classe dérivée. Deux éléments clés :

- une fonction `rotateVector()` qui tourne un vecteur d'un certain angle ;
- une liste d'angles prédéfinis (par exemple $\pm 15^\circ$, $\pm 30^\circ$).

```
1 static sf::Vector2f rotateVector(const sf::Vector2f& v, float degrees
   ) {
2     float rad = degrees * 3.14159265f / 180.f;
3     float cs  = std::cos(rad);
4     float sn  = std::sin(rad);
5
6     return sf::Vector2f(
7         v.x * cs - v.y * sn,
8         v.x * sn + v.y * cs
9     );
10 }
```

À partir du vecteur principal `toTarget` (de la tourelle vers la cible), nous appliquons successivement plusieurs rotations, ce qui produit autant de directions légèrement différentes. Pour chaque direction, on crée un projectile :

```
1 for (float a : angles) {
2     sf::Vector2f dir      = rotateVector(toTarget, a);
3     sf::Vector2f targetPos = getPosition() + dir;
4
5     outProjectiles.emplace_back(
6         getPosition(),
7         targetPos,
8         projectileSpeed,
9         damage,
10        4.f,          // rayon
11        allowedMask // m me masque que la tourelle
12    );
13 }
```

Dans notre implémentation, cela génère cinq projectiles (un central, deux à $\pm 15^\circ$, deux à $\pm 30^\circ$). Chaque projectile suit ensuite sa propre trajectoire, ce qui donne un vrai cône de tir visuel et utile en gameplay.

Cette approche est assez simple à mettre en œuvre, mais reste entièrement codée “en dur” : les angles sont figés dans le code et le nombre de projectiles aussi. Pour un jeu plus poussé, on préférerait sans doute des paramètres éditables (par exemple dans un fichier de config ou une interface d'édition).

5.7 Conclusion

En pratique, le comportement interne des tourelles et celui des ennemis spéciaux sont très proches : on applique un masque de filtrage, on teste la portée, on choisit une cible en fonction de la distance, puis on déclenche une action (tir ou attaque). La mécanique a été pensée d’abord côté tourelles, puis reprise côté ennemis en conservant la même logique.

Cette symétrie a plusieurs avantages :

- **réutilisation du code et des idées** : pas besoin d’inventer une nouvelle logique pour chaque camp ;
- **cohérence du comportement** : si on comprend le tir d’une tourelle, on comprend aussi celui d’un `FlyEnemy` ;
- **maintenance simplifiée** : toute évolution de la mécanique de tir peut se réfléchir de la même façon des deux côtés.

L’ensemble du système reste perfectible : le paramétrage des tourelles est encore très “statique” dans le code, et certaines fonctions (comme celles du shotgun) pourraient être généralisées. Mais à l’échelle de notre projet, le module `Tourelle` remplit son rôle : il fournit une base claire, extensible et suffisamment expressive pour construire un gameplay de défense varié.

6 Évolution du système de projectiles

Cette partie est consacrée au module `Projectile`, qui fait le lien direct entre les tourelles et les ennemis. Sans projectiles, les tourelles ne feraient que regarder les ennemis passer, et les ennemis « spéciaux » ne pourraient pas réellement menacer les défenses du joueur.

Le rôle de la classe `Projectile` est de représenter un tir simple : une entité qui part d'un point, se déplace en ligne droite dans une direction donnée, puis inflige des dégâts à une cible si elle la touche. Nous présentons ici :

- l'évolution du système de tir dans le projet ;
- la construction d'un projectile (position, vitesse, dégâts, rayon, cible) ;
- la gestion de sa trajectoire (normalisation, déplacement, suppression) ;
- la gestion des collisions (cercle/cercle) ;
- l'utilisation d'un masque (`EnemyMask`) pour filtrer les types d'ennemis ;
- son intégration globale dans le gameplay.

Globalement, la création de la classe `Projectile` a été assez naturelle : le fichier faisait déjà partie du squelette du projet, et son développement s'est fait en parallèle de `tryShoot()` et `acquireTarget()`. Les projectiles étant indispensables au fonctionnement des tourelles (puis plus tard des ennemis), il n'y a pas vraiment eu de phase « prototype » : nous avons plutôt enrichi la classe au fur et à mesure des besoins.

Dès le départ, nous avons listé les contraintes principales :

- les tourelles créent des projectiles pour attaquer les ennemis ;
- certains ennemis (`FlyEnemy` par exemple) créent aussi des projectiles pour viser les tourelles ;
- chaque projectile doit savoir **qui** il est censé toucher (ennemis ou tourelles) ;
- la collision doit tenir compte du rayon du projectile et de la taille de la cible.

6.1 Construction d'un projectile

La construction d'un projectile prend plusieurs paramètres :

- la position initiale et une position cible (`start`, `target`);
- la vitesse (`speed`);
- les dégâts (`damage`);
- le rayon (`radius`);
- un masque d'ennemis autorisés (`EnemyMask`);
- un type de cible (`TargetType` : ennemis ou tourelles).

```
1 // projectile.hpp
2
3 class Projectile {
4 public:
5     using EnemyMask = unsigned;
6
7     enum class TargetType {
8         Enemies,    // projectile destine aux ennemis
9         Towers      // projectile destine aux tourelles
10    };
11
12    Projectile(const sf::Vector2f& start,
13              const sf::Vector2f& target,
14              float speed          = 450.f,
15              int damage           = 0,
16              float radius         = 4.f,
17              EnemyMask allowedMask = ~EnemyMask{0},
18              TargetType targetType = TargetType::Enemies);
19 };
```

Dès l'initialisation, le projectile connaît donc son point de départ, sa direction, sa cible logique (ennemis ou tourelles), sa taille, sa vitesse et le type d'ennemis qu'il a le droit de toucher.

6.2 Normalisation de la direction

Avant de déplacer le projectile, il faut calculer sa direction. Pour cela, nous utilisons une fonction utilitaire `normalized()` qui renvoie un vecteur unitaire de même direction :

```
1 // projectile.cpp
2
3 static sf::Vector2f normalized(const sf::Vector2f& v) {
4     float n = std::sqrt(v.x*v.x + v.y*v.y);    // longueur du
        vecteur
5     return (n > 0.f) ? sf::Vector2f(v.x/n, v.y/n)
6                       : sf::Vector2f(0.f,0.f);
7 }
```

L'idée est d'éviter que la vitesse effective dépende de la distance entre `start` et `target`. Sans normalisation, un projectile tiré sur une cible lointaine irait beaucoup trop vite, et un projectile visant une cible très proche serait anormalement lent.

Dans le constructeur, nous calculons donc une vitesse cohérente :

```
1 Projectile::Projectile(const sf::Vector2f& start,
2                        const sf::Vector2f& target,
3                        float speed,
4                        int damage,
5                        float radius,
6                        EnemyMask allowedMask,
7                        TargetType targetType)
8     : dmg(damage),
9       alive(true),
10      allowedMask_(allowedMask),
11      targetType_(targetType)
12 {
13     shape.setRadius(radius);
14     shape.setOrigin(radius, radius);
15     shape.setFillColor(sf::Color::Yellow);
16     shape.setPosition(start);
17
18     // direction normalisee
19     auto dir = normalized(target - start);
20     // vitesse = direction * speed
21     velocity = dir * speed;
22 }
```

Cette approche est simple mais efficace : tous les projectiles se déplacent à la même vitesse théorique, quelle que soit la distance initiale. En contrepartie, nous n'avons pas de « visée anticipée » : les tirs sont purement directionnels et n'essaient pas de prédire les mouvements futurs de la cible.

6.3 Type de cible et masque EnemyMask

Les projectiles sont utilisés à la fois par les tourelles et par certains ennemis. Chaque projectile doit donc savoir :

- s’il vise des ennemis ou des tourelles (`TargetType`);
- quels types d’ennemis il est autorisé à toucher (`EnemyMask + EnemyKind`).

Le champ `TargetType` joue le rôle de « camp » :

- `TargetType::Enemies` : projectile tiré par une tourelle, destiné à toucher les ennemis ;
- `TargetType::Towers` : projectile tiré par un ennemi, destiné à toucher les tourelles.

Lorsqu’un projectile vise un ennemi, il doit en plus vérifier que ce type est bien autorisé par `allowedMask_`. C’est exactement le même principe que pour les tourelles : la fonction `accepts()` teste si le bit associé au type de l’ennemi est présent dans le masque.

```
1 bool Projectile::accepts(const Enemy* e) const noexcept {
2     if (!e) return false; // securite
3     const EnemyKind k = e->getKind(); // type reel de l'
        ennemi
4     return (allowedMask_ & kindBit(k)) != 0; // test du bit dans le
        masque
5 }
```

Cela permet de garantir, par exemple, que :

- les tirs de `FlyTourelle` ne détruisent que les ennemis volants ;
- les tourelles classiques ne touchent pas les ennemis volants si ce n’est pas prévu.

Ce mécanisme reste volontairement minimaliste (un simple masque binaire), mais il est suffisant pour notre palette actuelle de types d’ennemis. Avec davantage de variété (effets de statut, résistances, etc.), un système plus riche serait sans doute nécessaire.

6.4 Mise à jour et cycle de vie

Le cycle de vie d'un projectile est volontairement très simple : tant qu'il est « vivant », il se déplace ; dès qu'il sort trop loin de la zone de jeu ou qu'il touche une cible, il est marqué comme « mort » et sera ensuite retiré du vecteur de projectiles.

6.5 Mise à jour du mouvement

```
1 void Projectile::update(float dt) {
2     if (!alive) return; // projectile deja mort
3     // on ne fait rien
4
5     // Deplacement = vitesse * temps ecoule
6     shape.move(velocity * dt);
7
8     // On recupere la position pour verifier s'il est trop loin
9     auto p = shape.getPosition();
10
11     // Petites bornes pour dire : la c'est bon, on peut le supprimer
12     if (p.x < -100 || p.y < -100 || p.x > 5000 || p.y > 5000) {
13         alive = false;
14     }
15 }
```

Le mouvement est basé sur le vecteur `velocity` et le `dt` de la boucle de jeu, ce qui rend le déplacement indépendant du framerate. Les bornes « magiques » (-100 / 5000) sont suffisantes pour notre prototype, mais restent clairement perfectibles : dans une version plus générique, il serait préférable de s'appuyer sur la taille réelle de la carte.

6.6 Rendu graphique

```
1 void Projectile::draw(sf::RenderTarget& win) const {
2     if (alive) win.draw(shape);
3 }
```

Un projectile mort n'est plus dessiné. La suppression définitive est gérée ailleurs.

6.7 Gestion de l'état alive

L'attribut `alive` joue le rôle de petit masque logique :

- tant qu'il vaut `true`, `update()` et `draw()` s'appliquent normalement ;
- dès qu'il passe à `false`, le projectile est ignoré jusqu'à sa suppression.

On retrouve la même philosophie que pour `dead` / `reachedGoal` dans le module `Enemy` : séparer la logique de vie/mort de la gestion mémoire, pour garder une boucle de mise à jour plus lisible.

6.8 Gestion des collisions

La fonction `collidesWith()` teste si un projectile entre en contact avec une cible modélisée par un cercle (typiquement un ennemi). Pour cela, nous calculons la distance au carré entre les deux centres, et nous la comparons au carré de la somme des rayons :

```
1 bool Projectile::collidesWith(const sf::Vector2f& enemyPos ,
2                               float enemyRadius) const {
3     auto p = shape.getPosition();           // position du
        projectile
4     auto dx = p.x - enemyPos.x;             // ecart en X
5     auto dy = p.y - enemyPos.y;             // ecart en Y
6     float r = shape.getRadius() + enemyRadius; // rayon total
7
8     // Pythagore sur dx, dy mais sans racine
9     return (dx*dx + dy*dy) <= (r*r);
10 }
```

En évitant la racine carrée, on limite le coût des calculs tout en conservant une détection de collision parfaitement suffisante pour ce type de jeu. Là encore, le choix est simple mais adapté à notre besoin (pas de formes complexes, pas de hitbox multiples).

6.9 Conclusion

Le module **Projectile** joue un rôle central dans notre Tower Defense : il convertit les décisions de tir (tourelles et ennemis spéciaux) en effets concrets sur le terrain (dégâts, destruction des cibles, défense de la base).

Techniquement, la classe reste volontairement compacte : un cercle, une vitesse, un montant de dégâts, un masque d'autorisation, un type de cible et un booléen **alive**. Cette simplicité est compensée par une bonne intégration avec le reste du moteur :

- la normalisation de la direction garantit un comportement cohérent quelle que soit la cible ;
- le système **EnemyMask** réutilise la même logique de filtrage que les tourelles ;
- le flag **alive** et les bornes de la carte simplifient la gestion des ressources.

Du point de vue gameplay, mutualiser **Projectile** entre tourelles et ennemis spéciaux permet de garder une seule logique de tir : même façon de se déplacer, de toucher et d'infliger des dégâts. C'est plus facile à maintenir et cela ouvre la voie à des évolutions possibles (projectiles de zone, effets de statut, projectiles guidés, etc.) sans devoir repartir de zéro.

En résumé, même si la classe **Projectile** est plus compacte que **Enemy** ou **Tourelle**, elle constitue une brique essentielle de la boucle de combat et relie directement l'intention (tirer) et le résultat (toucher et détruire).

7 Game (boucle principale logique)

La classe **Game** représente le cœur logique du Tower Defense. Elle ne gère pas directement le rendu SFML (fenêtre, boucle **while**), mais orchestre tout ce qui fait vivre la partie :

- génération et destruction des ennemis et des tourelles ;
- mise à jour des entités (pathfinding, tirs, collisions, nettoyage) ;
- gestion des projectiles ;
- gestion des vagues d'ennemis.

L'objectif de cette section est de décrire le rôle de **Game**, les principaux choix de conception, ainsi que quelques limites de notre approche actuelle.

Concrètement, **Game** :

- reçoit les collections d'ennemis, de tourelles et de cellules à mettre à jour ;
- centralise la logique « temps réel » (attaques, déplacements, suppression des entités mortes) ;
- maintient en interne un `std::vector<Projectile>` (stockage par valeur) ;
- pilote la progression des vagues via un petit système embarqué.

7.1 Génération des ennemis

Au départ, les ennemis apparaissaient simplement en appelant `generateEnemy` toutes les x secondes, avec un type choisi aléatoirement :

```
1 // Faire apparaître un ennemi choisi aléatoirement
2 int r = std::rand() % 5; // 0..4
3
4 switch (r) {
5     case 0: enemies.push_back(new BasicEnemy()); break;
6     case 1: enemies.push_back(new FastEnemy()); break;
7     case 2: enemies.push_back(new TankEnemy()); break;
8     case 3: enemies.push_back(new FlyEnemy()); break;
9     case 4: enemies.push_back(new TargetEnemy()); break;
10 }
```

Ce principe simple a été conservé, mais l'apparition a été contrainte à une zone précise de la grille. Nous avons décidé que les trois premières colonnes seraient réservées au spawn des ennemis : aucune tourelle ne peut y être placée.

À partir du tableau global de cellules, nous construisons donc un tableau filtré `spawnCells`, ne contenant que les cellules des colonnes 0, 1 et 2. Lors de la création d'un ennemi, nous choisissons une de ces cellules au hasard et positionnons l'ennemi en son centre. Si, pour une raison quelconque, aucune cellule valide n'est disponible, nous détruisons immédiatement l'ennemi tout juste créé pour éviter une fuite mémoire.

```
1 // Recherche des cellules de spawn (colonnes 0, 1, 2)
2 std::vector<const Render::Cell*> spawnCells;
3 for (const auto& c : cells) {
4     if (c.col < 3)
5         spawnCells.push_back(&c);
6 }
7
8 // S'assure : aucune cellule de spawn -> on supprime l'ennemi
9 if (spawnCells.empty()) {
10     std::cout << "Erreur : aucune cellule de spawn disponible !" <<
11         std::endl;
12     delete enemies.back();
13     enemies.pop_back();
14     return;
15 }
16
17 // Choisir au hasard une cellule de spawn
18 int idx = std::rand() % spawnCells.size();
19 const Render::Cell* spawnCell = spawnCells[idx];
20
21 // Positionner l'ennemi au centre de la cellule
22 enemies.back()->setPosition(spawnCell->center.x, spawnCell->center.y)
23 ;
```

Ce système reste assez basique (spawn purement aléatoire), mais il garantit au moins une zone d'entrée propre et lisible pour les vagues.

7.2 Génération des tourelles

La génération des tourelles est similaire, mais pilotée par les entrées clavier du joueur. Dans le module de rendu, nous associons à chaque touche un type de tourelle et son coût :

```
1 // Extrait du fichier RENDER
2 {"[A]_Basic",    BasicTourelle(0.f).getCost()},
3 {"[Z]_Poison",   PoisonTourelle(0.f).getCost()},
4 {"[E]_Shotgun",  shotgunTourelle(0.f).getCost()},
5 {"[R]_Target",   TargetTourelle(0.f).getCost()},
6 {"[T]_Fly",      FlyTourelle(0.f).getCost()}
```

Dans `GameEvent`, les touches du clavier fixent un `currentTowerType` :

```
1 // Extrait de GameEvent
2 case sf::Keyboard::A: currentTowerType = 0;
3                       renderInfo.setPreviewTowerType(0); break;
4 case sf::Keyboard::Z: currentTowerType = 1;
5                       renderInfo.setPreviewTowerType(1); break;
6 case sf::Keyboard::E: currentTowerType = 2;
7                       renderInfo.setPreviewTowerType(2); break;
8 case sf::Keyboard::R: currentTowerType = 3;
9                       renderInfo.setPreviewTowerType(3); break;
10 case sf::Keyboard::T: currentTowerType = 4;
11                       renderInfo.setPreviewTowerType(4); break;
```

La fonction `Game::generateTourelle` reçoit alors :

- la liste des tourelles et des cellules;
- le pathfinding A*;
- la liste des ennemis;
- la position du clic souris (x, y);
- le type de tourelle à générer;
- un pointeur vers le `Player` (pour déduire les ressources).

```
1 void Game::generateTourelle(
2     std::vector<Tourelle*>& tourelles,
3     std::vector<Render::Cell>& cells,
4     PathFinding_AStar& pathfinder,
5     std::vector<Enemy*>& enemies,
6     float x, float y,    // position du clic
7     int type,            // type de tourelle
8     Player* player       // ressources du joueur
9 )
```

La première étape consiste à trouver la cellule cliquée et à vérifier qu'elle est valide :

1. **Clic dans la grille** : si aucune cellule ne contient (x, y) , on quitte.

```
1 // Trouver la cellule cliqu e
2 auto it = std::find_if(cells.begin(), cells.end(),
3     [x, y](Render::Cell& c){ return c.bounds.contains(x, y); });
4
5 if (it == cells.end()) return; // clic hors grille
```

2. **Zones interdites** : on bloque les 3 premières colonnes (zone de spawn) et la dernière colonne (zone d'arrivée).

```
1 // Interdit : colonnes de spawn + dernière colonne
2 if (it->col < 3 || it->col == pathfinder.gridCols - 1) {
3     std::cout << "Impossible de placer une tourelle ici.\n";
4     return;
5 }
```

3. **Cellule déjà occupée** : si `turreted == true`, on refuse le placement.

```
1 if (it->turreted) {
2     std::cout << "Cellule " << it->id << " déjà occupée!\n";
3     return;
4 }
```

Ensuite, on vérifie la compatibilité avec les ennemis déjà présents :

1. **Pas d'ennemi sur la cellule** :

```
1 // Refus si un ennemi est déjà sur la cellule
2 for (auto* e : enemies) {
3     if (!e || e->isDead()) continue;
4     const sf::Vector2f& pos = e->getPosition();
5     if (it->bounds.contains(pos)) {
6         std::cout << "Impossible : un ennemi est présent!\n";
7         return;
8     }
9 }
```

2. **Ne pas bloquer complètement le pathfinding** : on marque temporairement le nœud A* comme occupé, on teste si un chemin reste possible, puis on annule si besoin.

```
1 // Marquer le node comme occupé
2 int index = it->row * pathfinder.gridCols + it->col;
3 pathfinder.Nodes[index].turreted = true;
4
5 // Vérifier si un chemin reste possible
6 sf::Vector2f entryPos = cells.front().center;
7 if (!pathfinder.IsPathStillPossible(entryPos)) {
8     std::cout << "Placement refusé : cela bloquerait le chemin.\n";
9     pathfinder.Nodes[index].turreted = false;
10    return;
11 }
```


Si toutes les conditions de placement sont remplies, on vérifie ensuite le coût de la tourelle et les ressources du joueur :

```
1 // Coût selon le type (0..4)
2 int towerCost = 0;
3 switch (type) {
4     case 0: towerCost = 10; break;
5     case 1: towerCost = 30; break;
6     case 2: towerCost = 40; break;
7     case 3: towerCost = 40; break;
8     case 4: towerCost = 50; break;
9 }
10
11 if (player->getRessources() < towerCost) {
12     std::cout << "Pas assez de ressources (" << towerCost << ").\n";
13     pathfinder.Nodes[index].turreted = false;
14     return;
15 }
16
17 player->reduceRessources(towerCost);
```

Enfin, on instancie la bonne tourelle, on la place au centre de la cellule et on demande aux ennemis de recalculer leur chemin :

```
1 // Cr ation de la tourelle
2 switch (type) {
3     case 0: tourelles.push_back(new BasicTourelle(Render::Cell::
4         cellSize * 0.4f)); break;
5     case 1: tourelles.push_back(new PoisonTourelle(Render::Cell::
6         cellSize * 0.4f)); break;
7     case 2: tourelles.push_back(new shotgunTourelle(Render::Cell::
8         cellSize * 0.4f)); break;
9     case 3: tourelles.push_back(new TargetTourelle(Render::Cell::
10         cellSize * 0.4f)); break;
11     case 4: tourelles.push_back(new FlyTourelle(Render::Cell::
12         cellSize * 0.4f)); break;
13     default: break;
14 }
15
16 // Placement et marquage
17 tourelles.back()->setPosition(it->center.x, it->center.y);
18 tourelles.back()->setGridCoords(it->col, it->row);
19 it->turreted = true;
20
21 // Demande un recalcul de chemin
22 for (auto* e : enemies) {
23     if (e && !e->isDead()) e->needRepath = true;
24 }
```

Avec le recul, une factorisation des co ts et des param tres de tourelles (via une structure de donn es plut t que des `switch`) rendrait cette partie plus  volutive.

7.3 Destruction des entités

La destruction des tourelles et des ennemis suit un schéma classique : on parcourt le vecteur, `delete`, on met le pointeur à `nullptr`, puis on `clear` le vecteur :

```
1 void Game::destroyTourelles(std::vector<Tourelle*>& tourelles) {
2     for (auto*& t : tourelles) {
3         delete t;    // destructeur virtuel
4         t = nullptr;
5     }
6     tourelles.clear();
7 }
8
9 void Game::destroyEnemy(std::vector<Enemy*>& enemies) {
10    for (auto*& e : enemies) {
11        delete e;    // destructeur virtuel
12        e = nullptr;
13    }
14    enemies.clear();
15 }
```

Pour les projectiles, la destruction se fait dans `Game::update` à l'aide du pattern `erase-remove_if`. Comme vu dans la partie `Projectile`, chaque projectile met son `alive` à `false` lorsqu'il sort de la carte ou touche une cible, puis :

```
1 projectiles.erase(
2     std::remove_if(projectiles.begin(), projectiles.end(),
3         [](const Projectile& pr){ return !pr.isAlive(); })
4     ,
5     projectiles.end()
6 );
```

Ce nettoyage automatique à chaque frame évite que le vecteur de projectiles ne grossisse indéfiniment.

7.4 Gestion des vagues

Les vagues d'ennemis sont gérées directement dans `Game`. Avec un peu plus de temps, une classe dédiée aurait sans doute été préférable, mais la solution actuelle reste lisible.

Une vague est décrite par :

- un nombre d'ennemis à faire apparaître;
- un intervalle entre deux apparitions;
- un délai entre deux vagues.

Nous utilisons une petite structure `Wave` et quelques variables internes :

```
1 struct Wave { int count; float interval; };
2
3 std::vector<Wave> waves = {
4     {10, 0.6f}, {12, 0.5f}, {18, 0.4f}, {18, 0.4f}, {18, 0.4f}
5 };
6
7 int waveIndex = -1; // -1 = aucune vague démarrée
8 int toSpawn = 0; // ennemis restants spawner
9 float spawnIv = 0.f; // intervalle entre spawns
10 float spawnT = 0.f; // timer cumulatif
11 bool interWave = false; // délai entre deux vagues
12 float interT = 0.f; // timer d'inter-vague
13 float interDelay = 3.f; // délai entre vagues
14 bool wavesFinished = false;
```

`startWaves()` initialise le système, puis `updateWaves(dt, enemies, cells)` est appelée à chaque frame. Sa logique peut se résumer ainsi :

- si toutes les vagues sont terminées ou non démarrées, on quitte;
- si on est en phase inter-vague, on attend `interDelay` secondes, puis on passe à la vague suivante;
- sinon :
 - s'il reste des ennemis à spawner (`toSpawn > 0`), on les génère toutes les `spawnIv` secondes;
 - une fois tous les ennemis spawnés, on attend que le terrain soit vide pour lancer la phase inter-vague suivante.

Extrait de la phase inter-vague :

```
1 // Phase inter-vague
2 if (interWave) {
3     interT += dt;
4     if (interT >= interDelay) {
5         interWave = false;
6         interT = 0.f;
7         ++waveIndex;
8         if (waveIndex >= (int)waves.size()) {
9             wavesFinished = true;
10            std::cout << "Toutes les vagues sont termin es!\n";
11            return;
12        }
13
14        toSpawn = waves[waveIndex].count;
15        spawnIv = waves[waveIndex].interval;
16        spawnT = 0.f;
17        std::cout << "Vague " << (waveIndex+1) << "!\n";
18    }
19    return;}
```

Extrait de la phase de spawn :

```
1 // Phase de spawn
2 if (toSpawn > 0) {
3     spawnT += dt;
4     while (toSpawn > 0 && spawnT >= spawnIv) {
5         generateEnemy(enemies, cells);
6         --toSpawn;
7         spawnT -= spawnIv;
8     }
9     return;
10 }
```

Enfin, lorsqu'il n'y a plus d'ennemis vivants, on déclenche la prochaine phase inter-vague :

```
1 // Attendre que le terrain soit vide
2 bool encoreDesEnnemis = false;
3 for (auto* e : enemies) {
4     if (e && !e->isDead()) {
5         encoreDesEnnemis = true;
6         break;
7     }
8 }
9 if (!encoreDesEnnemis) {
10     interWave = true;
11     interT = 0.f;
12     std::cout << "Vague " << (waveIndex+1)
13         << " termin e .\nProchaine dans "
14         << interDelay << "s.\n";
15 }
```

Le système reste simple, entièrement codé en dur, mais suffisant pour proposer une progression basique des vagues.

7.5 La boucle principale Game::update

Game::update est appelée à chaque frame et sert de point central à toute la logique du jeu : déplacements, tirs, collisions, nettoyage, gestion de fin de partie. L'idée est d'avoir un seul endroit où coordonner les modules Enemy, Tourelle, Projectile, PathFinding_AStar et Player.

La séquence générale est la suivante :

1. Donner la liste des tourelles aux ennemis spéciaux.

Certains ennemis (FlyEnemy, TargetEnemy) ont besoin de connaître les tourelles pour pouvoir les viser. En début de update, on utilise un dynamic_cast pour détecter ces types et leur passer un pointeur vers le vecteur de tourelles.

```
1  for (auto* e : enemies) {
2      if (!e) continue;
3
4      if (auto* f = dynamic_cast<FlyEnemy*>(e)) {
5          f->setTowerList(&tourelles);
6      } else if (auto* t = dynamic_cast<TargetEnemy*>(e)) {
7          t->setTowerList(&tourelles);
8      }
9  }
```

2. Mettre à jour les ennemis et gérer leur mort / arrivée.

On parcourt ensuite le vecteur enemies : pour chaque ennemi, on appelle update(pathfinder, cells), puis on vérifie s'il a atteint l'objectif ou s'il est mort. Dans le premier cas, le joueur perd de la vie; dans le second, il gagne des ressources.

```
1  for (auto it = enemies.begin(); it != enemies.end(); ) {
2      Enemy* e = *it;
3      if (!e) { it = enemies.erase(it); continue; }
4
5      e->update(pathfinder, cells);
6
7      if (e->reachedGoal) {
8          player->reduceHealth(e->getDamage());
9          delete e;
10         it = enemies.erase(it);
11     } else if (e->isDead()) {
12         player->AddRessources(e->ressourceValue);
13         delete e;
14         it = enemies.erase(it);
15     } else {
16         ++it;
17     }
18 }
```

3. Tirs des tourelles puis des ennemis spéciaux.

Les tourelles actives tirent d'abord sur les ennemis :

```
1 for (auto* t : tourelles) {
2     if (!t || t->isDestroyed()) continue;
3     t->tryShoot(dt, enemies, projectiles);
4 }
```

Puis les ennemis spéciaux tirent sur les tourelles, via une méthode symétrique :

```
1 for (auto* e : enemies) {
2     if (!e || e->isDead()) continue;
3
4     if (auto* f = dynamic_cast<FlyEnemy*>(e)) {
5         f->tryShoot(dt, projectiles);
6     } else if (auto* t = dynamic_cast<TargetEnemy*>(e)) {
7         t->tryShoot(dt, projectiles);
8     }
9 }
```

4. Avancer tous les projectiles.

```
1 for (auto& p : projectiles) {
2     p.update(dt);
3 }
```

5. Collisions projectiles / ennemis.

Pour chaque projectile « côté tourelles », on teste la collision avec les ennemis autorisés (via `accepts()` et le masque `EnemyMask`) :

```
1 for (auto& p : projectiles) {
2     if (!p.isAlive()) continue;
3
4     for (auto* e : enemies) {
5         if (!e || e->isDead()) continue;
6         if (!p.accepts(e)) continue;
7
8         const float enemyRadius = e->getRadius();
9         if (p.collidesWith(e->getPosition(), enemyRadius)) {
10             e->takeDamage(p.getDamage());
11             p.kill();
12             break;
13         }
14     }
15 }
```

6. Collisions projectiles / tourelles et suppression des tourelles détruites.

On applique la même logique pour les projectiles visant les tourelles :

```
1  for (auto& p : projectiles) {
2      if (!p.isAlive()) continue;
3      if (p.getTargetType() != Projectile::TargetType::Towers)
4          continue;
5
6      for (auto* t : tourelles) {
7          if (!t || t->isDestroyed()) continue;
8
9          const float towerRadius = t->getRadius();
10         if (p.collidesWith(t->getPosition(), towerRadius)) {
11             t->takeDamage(p.getDamage());
12             p.kill();
13             break;
14         }
15     }
```

Ensuite, on supprime les tourelles détruites et on libère leur cellule dans la grille et dans A* :

```
1  for (auto it = tourelles.begin(); it != tourelles.end(); ) {
2      Tourelle* t = *it;
3
4      if (!t || t->isDestroyed()) {
5          if (t) {
6              int col = t->getGridCol();
7              int row = t->getGridRow();
8
9              if (col >= 0 && row >= 0) {
10                 auto cellIt = std::find_if(
11                     cells.begin(), cells.end(),
12                     [col, row](const Render::Cell& c) {
13                         return c.col == col && c.row == row;
14                     }
15                 );
16                 if (cellIt != cells.end()) {
17                     cellIt->turreted = false;
18                     int index = cellIt->row * pathfinder.gridCols
19                         + cellIt->col;
20                     pathfinder.Nodes[index].turreted = false;
21                 }
22             }
23
24             delete t;
25             it = tourelles.erase(it);
26         } else {
27             ++it;
28         }
29     }
```

7. Nettoyage des projectiles et fin de partie.

On termine par le nettoyage des projectiles morts (pattern `erase-remove_if`) :

```
1  projectiles.erase(  
2      std::remove_if(projectiles.begin(), projectiles.end(),  
3          [](const Projectile& pr){ return !pr.isAlive()  
4              ; } ),  
5      projectiles.end()  
6  );
```

Enfin, on vérifie la vie du joueur. Si elle tombe à 0, on passe `gameOver` à `true` :

```
1  if (player->getHealth() <= 0) {  
2      player->reduceHealth(0); // force    0  
3      gameOver = true;  
4      std::cout << "GAME OVER!" << std::endl;  
5  }
```

Au final, `Game::update` joue réellement le rôle de « chef d'orchestre » : elle ne contient pas toute la logique métier (la plupart est dans `Enemy`, `Tourelle`, `Projectile`), mais elle décide de l'ordre dans lequel tout s'exécute et des interactions entre modules. La conception de cette classe s'est imposée assez naturellement : on s'est vite rendu compte qu'il fallait un point central pour garder le gameplay cohérent. Avec davantage de temps, une séparation plus fine (par exemple une classe dédiée aux vagues) rendrait l'ensemble plus modulaire, mais dans l'état actuel du projet, `Game` remplit correctement son rôle.

8 Render

8.1 Introduction

Le module **Render** constitue l'ensemble des classes responsables de l'affichage dans le Tower Defense. Il s'appuie sur la bibliothèque graphique **SFML** et repose sur une architecture orientée objet organisée autour d'une classe principale (*Render*) et de plusieurs classes dérivées spécialisées pour chaque écran ou zone d'affichage du jeu.

Nous avons créé les rendus suivant :

- Render
- RenderMainMenu
- RenderMap
- RenderMenu
- RenderControl
- RenderInfo
- RenderGameOver

Tout d'abord nous avons créé la classe Render

8.2 Classe Principale : Render

La classe **Render** constitue la classe de base utilisée pour gérer le rendu graphique dans le projet. Elle fournit des outils permettant d'afficher une texture, de dessiner la grille utilisée par les autres composants du jeu, ainsi que de générer l'ensemble des cellules qui composent cette grille.

Cell

Pour créer notre jeu, nous avons organisé l'espace par une grille de cellule.

Définissons Cell une structure comportant :

- un identifiant — id
- une colonne qui lui est assigné pour sa "position" suivant l'axe X — col
- une ligne pour sa "position" suivant l'axe Y — row
- une forme — sf : :FloatRect bounds
- un vecteur contenant sa position (X,Y) — center
- un booléen — turreted
- une dimension — cellSize

Le booléen "turreted" informe si la cellule est occupé par une tour.
cellSize est une valeur flottante statique. Ainsi toutes les cellules auront la même dimension

Fonctions propre à Render

Render possède des méthodes d'accès simples :

- `getTexture` retourne la `RenderTexture` interne,
- `getSprite` retourne le sprite qui encapsule cette texture,
- `getSize` renvoie la taille de la texture.

Ces méthodes sont utiles pour permettre à des classes dérivées ou à d'autres modules de récupérer les informations graphiques nécessaires qui sont protégés.

La méthode `buildCells` génère toutes les cellules qui composent la grille :

1. elle récupère la taille du `RenderTarget`,
2. elle calcule le nombre de colonnes et de lignes,
3. elle crée une cellule pour chaque combinaison (ligne, colonne),
4. elle attribue :
 - un identifiant unique,
 - les indices `row` et `col`,
 - un rectangle `bounds` représentant la zone physique,
 - les coordonnées du centre.

Chaque cellule est stockée dans un vecteur qui sera ensuite utilisé par les autres modules du jeu (pathfinding, placement des tourelles, sélection, etc.).

La méthode `drawGrid` trace une grille sur la fenêtre ou la texture cible. Elle utilise des lignes verticales et horizontales séparées par un espacement égal à `cellSize`.

Pour chaque ligne verticale :

- une ligne est tracée de haut en bas,
- la position en X augmente de `cellSize` à chaque itération.

Pour chaque ligne horizontale :

- une ligne est tracée de gauche à droite,
- la position en Y augmente de `cellSize` à chaque pas.

On obtient après construction des cellules la figure ci-dessous :

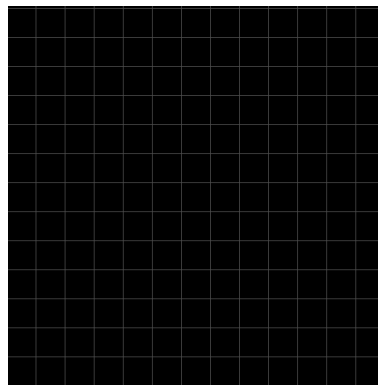


FIGURE 9 – Création et affichage de la répartition de la map (en cellules)

Fonctions virtuel de Render

La méthode `display()` appelle :

```
1 texture.display();
```

Cela indique que tous les dessins effectués sur la `RenderTexture` doivent être finalisés afin que le sprite correspondant puisse être mis à jour.

La méthode `update` est définie mais sera redéfini par les classes dérivées qui auraient besoin de mettre à jour leur rendu.

`drawTo (sf : :RenderWindow& target)` dessine le sprite final sur la fenêtre cible de jeu.

- Une `sf::RenderTexture` interne servant de surface de dessin indépendante.
- Un `sf::Sprite` associé permettant d’afficher la texture.
- Une structure interne **Cell** utilisée pour diviser la carte en une grille d’emplacements.

Cependant, pour mieux structurer notre jeu / optimiser la fenêtre de jeu, nous avons réparti cette fenêtre en 3 zones : `RenderMap`, `RenderInfo`, `RenderControl`.



FIGURE 10 – Répartition du Render en une zone de Map, d’Info et de Contrôle

8.3 Classe RenderMap

Le module `RenderMap` hérite de la classe `Render`. Cette classe gère l’affichage de la zone de jeu principale. On y affiche les ennemis, les tours, les projectiles. Pour un meilleur rendu un fond a été rajouté. Ce fond n’est que décoratif.

La fonction `Render : :drawGrid` est appelée pour dessiner les cellules. La grille contiendra les tourelles et les ennemis et un texte indiquant la vague actuelle (figure 2)



FIGURE 11 – RenderMap avec apparition d’ennemis

8.4 Classe `RenderInfo`

La classe `RenderInfo` dérive de la classe abstraite `Render`. Elle représente le panneau latéral droit de l'interface du jeu, chargé d'afficher les informations du joueur ainsi que celles d'une tourelle sélectionnée ou prévisualisée.

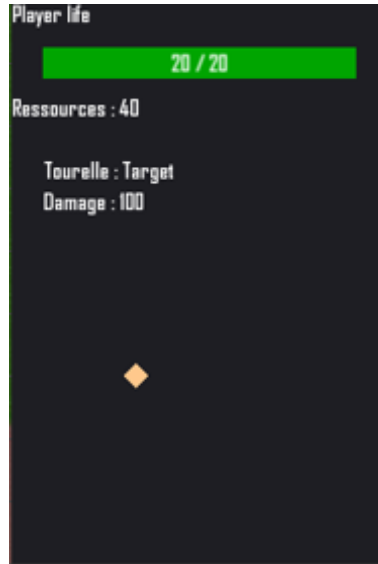


FIGURE 12 – Affichage informations Joueur et tourelle

Informations Joueur

Concernant le joueur, une barre affiche ses points vie. Lorsqu'un ennemi arrive au bout de la grille, le joueur prend des dégâts.

Pour visualiser cela, la valeur de vie diminue ainsi que la dimension de la barre de vie.

En dessous de 50 % et 25 % des points de vie, la barre passe du vert au orange, puis rouge.

De plus les ennemis rapportent au joueur des ressources à leur mort. Cette donnée est affichée pour montrer ce qu'il reste.

Informations Tourelle

Lorsque le joueur appuie sur une tour ou veut en créer une, les informations suivantes apparaissent :

- Nom de la tourelle.
- Dégâts.
- Prévisualisation graphique de la tourelle sélectionnée.

La prévisualisation est réalisé grâce à `cloneShape`, une méthode qui illustre la tour, peu importe sa forme. Cela aurait eu plus d'utilité si nos tours avaient différente forme.

`updateTowerPreview` met à jour toutes les informations d'une tourelle sélectionnée ou d'une tourelle en prévisualisation.

8.5 Classe `RenderControl`

La classe `RenderControl` hérite de la classe de base `Render` et représente le panneau d'information et de contrôle affiché à droite de l'écran.

Elle permet au joueur d'accéder à plusieurs onglets : gestion des tourelles, liste des ennemis et ouverture du menu du jeu. La méthode `handleClick` permet de détecter le clic du joueur.

Afin de guider le joueur sur les tours qui sont construisables, l'onglet "Tourelles" affiche :

- le nom des tourelles disponibles,
- leur coût en ressources,
- une couleur différente si le joueur ne peut pas se les payer
- les touches pour les construire

Les informations sont basées sur les objets tourelles instanciés temporairement pour obtenir leur coût via la méthode `getCost()`.

Les ennemis sont présents dans l'onglet "Ennemis". Seulement la liste des différents types d'ennemis est affiché.

L'onglet Menu était initialement un onglet pour afficher les touches de paramètre de jeu, quitter,...

Cependant cet onglet se comporte comme un bouton qui permettra de changer l'état du jeu.

8.6 Classe RenderMenu

Cette classe gère un menu semi-transparent affiché en jeu. Le joueur la fait apparaître avec l'onglet Menu du RenderControl.

Durant le menu le jeu est en pause et donne accès à trois bouton.

Le menu apparant, contient :

- Un titre : *MENU*.
- Un bouton **Play**.
- Un bouton **Restart**.
- Un bouton **Quit**.

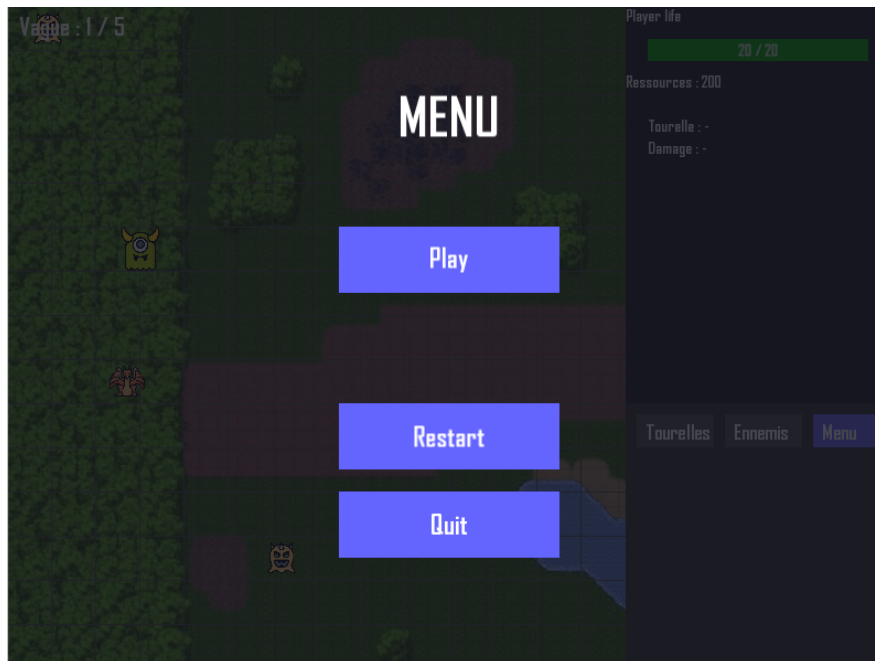


FIGURE 13 – Menu

Si le joueur appuie sur Play, il retourne dans sa partie. Si il clic sur Restart, le jeu recommence de 0. Quit lui permet de retourner dans le Menu Principale.

8.7 Classe RenderGameOver

Le joueur n'a plus de points de vie? Et ben un rendu "RenderGameOver" apparaît et affiche "Game Over".



FIGURE 14 – Game Over !

Comme le Menu, il y a le bouton Quit et Restart avec les même comportement. Tout se fige aussi car il n'y a plus d'intérêt à dérouler les autres vagues.

8.8 Classe RenderMainMenu

Comme tout jeu, une fenêtre d'accueil ou de présentation est un élément crucial. RenderMainMenu possède donc un fond d'écran et deux boutons pour soit lancer le jeu ou quitter et fermer le jeu.



FIGURE 15 – Ecran principal du jeu

9 Game Event

9.1 Introduction

La classe `GameEvent` gère l'ensemble des interactions entre le joueur, la fenêtre SFML, les menus du jeu, les tourelles, les ennemis, ainsi que les transitions entre les différents états de l'application (`AppState`).

9.2 L'énumération `AppState`

L'énumération `AppState` définit les quatre états possibles de l'application :

- **MainMenu** : écran principal du jeu,
- **Playing** : partie en cours,
- **Menu** : menu pause accessible en jeu,
- **GameOver** : écran de fin de partie.

Chaque état possède un gestionnaire dédié dans la classe `GameEvent`.

9.3 Rôle général de la classe `GameEvent`

La classe `GameEvent` fournit une méthode principale :

- **processEvents** : analyse tous les événements émis par SFML et renvoie le nouvel état de l'application.

Elle utilise pour cela plusieurs méthodes internes spécialisées :

- `handleMainMenu`,
- `handlePlaying`,
- `handleMenu`,
- `handleGameOver`.

Ces méthodes déterminent précisément les actions possibles selon l'état en cours. La méthode `processEvents` gère :

- la fermeture de la fenêtre,
- la touche Échap,
- puis oriente chaque événement vers le gestionnaire approprié selon l'état courant.

Le premier état est le `MainMenu` car c'est le rendu apparant lors du lancement du jeu.

9.3.1 Gestion de l'État MAIN MENU

La méthode `handleMainMenuEvents` défini pour la gestion de la fenêtre principale permet de :

- démarrer le jeu avec la touche Entrée,
- cliquer sur le bouton Start,

— fermer le programme avec Quit.

Elle initialise également les vagues et active la musique d'introduction. Si le joueur appuie sur Start, le jeu passe en mode "Playing".

9.3.2 Gestion de l'État PLAYING

L'état **Playing** correspond au mode principal du jeu.

La méthode `handlePlaying` gère plusieurs types d'interactions :

_ Changement de type de tourelle

Les touches A, Z, E, R, T sélectionnent un type de tourelle et mettent à jour l'aperçu dans `RenderInfo`. Ces touches sont indiquées dans l'onglet Tourelle du `RenderControl`.

_ Placement des tourelles

Pour placer une tourelle sur la map (`RenderMap`), il faut faire un clic gauche.

Avec l'évènement du clic, la fonction `generateTourelle` de la classe `Game` va vérifier des conditions pour poser une tourelle.

Le clic droit pour sélectionner la tourelle et affiche dans `RenderInfo` ses informations (dégâts, forme, type).

La molette, elle permet d'améliorer les tours pour augmenter leur puissance.

_ Affichage du menu

Le panneau latéral (`RenderControl`) réagit aux clics pour changer l'état du jeu.

Etat actuel : Menu

Clic sur le bouton :

Play → *Playing*

Restart → *Playing*

Quit → *MainMenu*

Sélection d'une tourelle

Un clic droit sélectionne une tourelle existante, dont les informations sont ensuite affichées dans `RenderInfo`.

9.3.3 Gestion de l'État MENU (Pause)

La fonction `handleMenu` gère les trois boutons du menu pause :

- **Play** : retour au jeu,
- **Restart** : réinitialisation complète,
- **Quit** : retour au menu principal.

Elle contrôle également la musique en fonction de la transition vers un nouvel état.

9.3.4 Gestion de l'État GAME OVER

L'écran game over passe l'état du jeu à `GameOver`. Il propose aussi deux choix :

- **Restart** : nouvelle partie -> `Playing`
- **Quit** : retour au menu principal -> `MainMenu`

La méthode `handleGameOver` interagit avec la musique et masque l'interface correspondante.

9.3.5 Méthode utilitaire : `resetGame`

Lorsque le joueur clic sur un des boutons Restart de Game Over ou de Menu mais aussi le bouton Quit de Menu, la méthode `resetGame` reconstruit entièrement l'état du jeu :

- destruction des ennemis et tourelles,
- réinitialisation du joueur,
- recréation du jeu,
- reconstruction des cellules via `Render`,
- recréation complète du pathfinding A*,
- réinitialisation des informations affichées,
- relance de la première vague.

10 MAIN

10.1 Introduction

Le main est notre programme principale de notre projet.
Ici la construction des classes, la gestion des éléments, leur affichage et l'appel des musiques sont faites.

10.2 Construction des classes

Nous avons besoin de construire les classes de rendu avec la dimension de notre fenêtre, le joueur, notre gestion du jeu (game), notre Pathfinding et nos .

```
— RenderMap renderMap(game_window.getSize());
— RenderControl renderControl(game_window.getSize());
— RenderInfo renderInfo(game_window.getSize());
— RenderGameOver renderGameOver(game_window.getSize());
— RenderMenu renderMenu(game_window.getSize());
— RenderMainMenu renderMainMenu(game_window.getSize());
=====

— Game game;
— Player player;
— PathFinding_AStar pathfinder;
— AppState appState = AppState : :MainMenu;
— GameEvent gameEvent(.....)
```

D'initialiser nos cellules, nos noeuds, notre temps (clock), notre période d'apparition.

10.3 Tant que la fenêtre est ouverte

Tant que la fenêtre est ouverte le jeu doit continuellement vérifier l'état de appState. Cela lui permet de créer, déplacer, ou non les ennemis, tourelles, projectils.

Si le jeu est en MainMenu, seulement les fonctionnalités de MainMenu son disponible (boutons et musique).

Sinon le jeu vérifie si le joueur a perdu (Game Over), si la fenêtre "Menu" est ouverte ou si l'état actuel est "Playing".

10.4 la fenêtre se ferme

Si le joueur ferme la fenêtre par le bouton Quitter du MainMenu ou appuit sur la croix de la fenêtre, nous libérons la mémoire et espérons qu'il rejouera...

11 Conclusion sur l'ensemble du projet

Le développement de ce projet a été extrêmement complexe. Travailler en binôme semblait, sur le papier, être une excellente idée : deux personnes, donc deux fois plus de productivité... mais la réalité est bien différente. Travailler ensemble demande bien plus que se répartir des tâches : il faut harmoniser nos façons de coder, nos habitudes, notre logique, nos conventions de nommage. Nous avons rapidement constaté que la fusion de nos deux parties devenait une source de ralentissement constante : variables renommées, philosophies de programmation différentes, incompréhensions, corrections en chaîne... Pourtant, il faut aussi reconnaître que sans ce binôme, nous n'aurions probablement pas été aussi loin. Cette expérience nous a montré que le travail d'équipe, qu'on pense souvent "acquis", est en réalité bien plus difficile qu'il n'y paraît.

Le projet nous a demandé un temps colossal. Beaucoup trop, pour être honnête, au vu de l'ensemble des autres projets très exigeants que nous avons en parallèle, sans compter les révisions. Même si créer un jeu est passionnant et très formateur en C++, cela reste un travail immense, et nous avons dû faire de nombreux sacrifices pour mener ce projet jusqu'au bout.

Au début, nous voulions vraiment faire "notre" jeu, sans aide extérieure. Pendant plusieurs semaines, nous avons refusé d'utiliser toute IA générative. Avec le recul, cette décision nous a mis un retard énorme que nous n'avons rattrapé qu'au prix de journées (et surtout de nuits) entières passées sur le projet. Notre manque initial de compétences en C++ n'a évidemment pas aidé : c'était un langage encore neuf pour nous, et chaque avancée nécessitait du temps, de la recherche et beaucoup d'essais-erreurs.

Nous aurions aimé produire un jeu dont nous étions réellement fiers : complet, propre, fidèle à toutes nos idées. Mais le temps, nos limites techniques du moment et l'ampleur du projet nous ont rattrapés. Le jeu final fonctionne, c'est un Tower Defense jouable, mais il ne représente qu'une infime partie de ce que nous voulions réaliser.

Malgré tout, nous avons énormément appris. Ce projet a été une vraie porte d'entrée vers le C++ et vers le développement d'un jeu sans moteur externe.

(Enzo) : Ce projet m'a donné envie de reprendre tout depuis zéro, pour créer un autre jeu personnel, entièrement codé en C++ sans moteur comme Unity ou Unreal. J'ai beaucoup aimé le langage, et même si le projet actuel laisse un goût de frustration, il m'a motivé pour aller plus loin.

Enfin, il existe de nombreuses pistes d'amélioration : supprimer les éléments inutiles, nettoyer certaines variables, soigner davantage l'aspect visuel, créer un système d'amélioration plus abouti, développer un éditeur de map, un mode debug, ou même une sandbox de test. Le potentiel est là, il ne manque que du temps... et surtout de l'expérience.

(Alexandre) : Ce jeu est le meilleur et en plus GRATUIT !